MODULE *hovercraft*

Based on the formal specification for the Raft consensus algorithm.
We introduce the *Hovercraft Switch* to verify safety properties.
We track message counts for advancing entry commit index.
Modified by *Ovidiu Marcu*.

In standard Raft the leader is the central hub.
A client sends a request only to the leader.
The leader must then replicate the entire request *payload* to all followers
The leader gathers acknowledgments, commits the entry, applies it,
and sends a response to the client.
Bottleneck: The leader's network bandwidth and processing power for sending
the full *payload* to every follower becomes a limiting factor as the
cluster size (N) or request size increases.
Throughput is limited by $Leader\_Bandwidth$ / $((N-1) * Request\_Payload\_Size)$.

We introduce a "Switch" abstraction (representing mechanisms like *IP* Multicast
or a dedicated middlebox/programmable switch as used in the *HovercRaft* paper)
First Requirement: Client sends via *Switch*.
The *Switch*'s responsibility is to deliver the request *payload* to all server
nodes (Leader and Followers) "simultaneously".
Second Requirement: *Leader* orders via entry *Metadata*
The leader still receives the client request *payload*
(via the *Switch*, just like followers).
Leader's primary role shifts from ordering and *payload* replication to just ordering.
The leader sends only a fixed-size metadata message to the followers.
This decouples the ordering traffic from the request *payload* size.
The leader's outbound traffic for consensus becomes $(N-1) * Metadata\_Size$,
which is much smaller than replicating the full *payload*.
Third Requirement: Followers match *payload* and *Metadata*
Followers receive the raw request payloads directly from the *Switch*.
Since network delivery (especially multicast) might lose data,
followers must buffer these incoming requests temporarily.
When a follower receives the ordering metadata message from the leader,
it uses the identifier in the metadata to find the corresponding *payload*
in its temporary buffer.
Once matched, the follower places the request *payload* into its replicated *log*
at the correct index specified by the leader's metadata.

EXTENDS *Naturals*, *FiniteSets*, *Sequences*, *TLC*

********** CONSTANTS **********

The set of server *IDs* including the *Switch*
CONSTANTS *Server*

The set of client requests that can go into the *log*
CONSTANTS *Value*

Server states.
CONSTANTS *Follower*, *Candidate*, *Leader*, *Switch*

A reserved value.
CONSTANTS *Nil*

Message types:
CONSTANTS *RequestVoteRequest*, *RequestVoteResponse*,
AppendEntriesRequest, *AppendEntriesResponse*

For instrumentation to limit model state space
CONSTANTS *MaxClientRequests*

Maximum times a server can become a leader
CONSTANTS *MaxBecomeLeader*

Maximum term number allowed in the model
CONSTANTS *MaxTerm*

********** VARIABLES **********

Global variables

A bag of records representing requests and responses sent from one server
to another. This is a function mapping Message to *Nat*.
VARIABLE *messages*

Counter for how many times each server has become leader
VARIABLE *leaderCount*

maximum client requests so far
VARIABLE *maxc*

variable for tracking entry commit message counts
Maps $\langle logIndex, logTerm \rangle$ to a record tracking message counts.
[*sentCount* $\mapsto$ *Nat*, * *AppendEntriesRequests* sent for the entry
ackCount $\mapsto$ *Nat*, * *AppendEntriesResponses* received for this entry
committed $\mapsto$ *Bool*] * Flag indicating if the entry is committed
VARIABLE *entryCommitStats*

$instrumentationVars \triangleq \langle leaderCount, maxc, entryCommitStats \rangle$

index into *Server*'s *Switch*
VARIABLE *switchIndex*

Storage for requests received by the switch before they're replicated
Maps request value to the full *payload* entry
VARIABLE *switchBuffer*

Each server's buffer of unordered requests received from the *Switch*
Maps from *Server* to a set of request values pending ordering
VARIABLE *unorderedRequests*

Records which $\langle value, term \rangle$ pairs the *Switch* has sent to each server.
Maps *Server ID* $\rightarrow$ Set of $\langle$ *Value*, *Term*$\rangle$ pairs.
VARIABLE *switchSentRecord*

New *HovercRaft* variables

$$hovercraftVars \triangleq \langle switchBuffer, unorderedRequests, switchIndex, switchSentRecord \rangle$$

The following variables are all per server (functions with domain *Server*).

The server's term number.
VARIABLE *currentTerm*

The server's state (Follower, *Candidate*, or *Leader*).
VARIABLE *state*

The candidate the server voted for in its current term, or
Nil if it hasn't voted for any.
VARIABLE *votedFor*

$$serverVars \triangleq \langle currentTerm, state, votedFor \rangle$$

A Sequence of *log* entries. The index into this sequence is the index of the *log* entry.
VARIABLE *log*

The index of the latest entry in the *log* the state machine may apply.
VARIABLE *commitIndex*

$$logVars \triangleq \langle log, commitIndex \rangle$$

The following variables are used only on candidates:
The set of servers from which the candidate has received a *RequestVote* response in its *currentTerm*.
VARIABLE *votesResponded*

The set of servers from which the candidate has received a vote in its *currentTerm*.
VARIABLE *votesGranted*

A history variable used in the proof. This would not be present in an

implementation.
Function from each server that voted for this candidate in its *currentTerm*
to that voter's *log*.
VARIABLE $voterLog$

$candidateVars \triangleq \langle votesResponded, votesGranted, voterLog \rangle$

The following variables are used only on leaders:
The next entry to send to each follower.
VARIABLE $nextIndex$

The latest entry that each follower has acknowledged is the same as the
leader's. This is used to calculate *commitIndex* on the leader.
VARIABLE $matchIndex$

$leaderVars \triangleq \langle nextIndex, matchIndex \rangle$

The set of server *IDs* excluding the current *Switch* server.
$Servers \triangleq Server \setminus \{switchIndex\}$

VARIABLE $Servers$

All variables; used for stuttering (asserting state hasn't changed).
$vars \triangleq \langle messages, serverVars, candidateVars, leaderVars, logVars,$
$instrumentationVars, hovercraftVars, Servers \rangle$

********** HELPERS **********

The set of all quorums. This just calculates simple majorities, but the only
important property is that every quorum overlaps with every other.
$Quorum \triangleq \{i \in \text{SUBSET } (Servers) : Cardinality(i) * 2 > Cardinality(Servers)\}$

The term of the last entry in a *log*, or 0 if the *log* is empty.
$LastTerm(xlog) \triangleq \text{IF } Len(xlog) = 0 \text{ THEN } 0 \text{ ELSE } xlog[Len(xlog)].term$

$WithMessage(m, msgs) \triangleq$
IF $m \in$ DOMAIN $msgs$ THEN
$msgs$
ELSE
$msgs$ @@ $(m :> 1)$

$WithoutMessage(m, msgs) \triangleq$
IF $m \in$ DOMAIN $msgs$ THEN
$[msgs$ EXCEPT $![m] = \text{IF } msgs[m] > 0 \text{ THEN } msgs[m] - 1 \text{ ELSE } 0]$
ELSE
$msgs$

Add a message to the bag of messages.
$Send(m) \triangleq messages' = WithMessage(m, messages)$

```
Remove a message from the bag of messages. Used when a server is done
processing a message.
Discard(m) ≜ messages' = WithoutMessage(m, messages)

Helper for Send and Reply. Given a message m and bag of messages, return a
Combination of Send and Discard
Reply(response, request) ≜
    messages' = WithoutMessage(request, WithMessage(response, messages))

Return the minimum value from a set, or undefined if the set is empty.
Min(s) ≜ CHOOSE x ∈ s : ∀ y ∈ s : x ≤ y

Return the maximum value from a set, or undefined if the set is empty.
Max(s) ≜ CHOOSE x ∈ s : ∀ y ∈ s : x ≥ y

min(a, b) ≜ IF a < b THEN a ELSE b

ValidMessage(msgs) ≜
    {m ∈ DOMAIN messages : msgs[m] > 0}

The prefix of the log of server i that has been committed up to term x
CommittedTermPrefix(i, x) ≜
Only if log of i is non-empty, and if there exists an entry up to the term x
    IF Len(log[i]) ≠ 0 ∧ ∃ y ∈ DOMAIN log[i] : log[i][y].term ≤ x
     THEN
then, we use the subsequence up to the maximum committed term of the leader
      LET maxTermIndex ≜
            CHOOSE y ∈ DOMAIN log[i] :
              ∧ log[i][y].term ≤ x
              ∧ ∀ z ∈ DOMAIN log[i] : log[i][z].term ≤ x  ⇒ y ≥ z
      IN  SubSeq(log[i], 1, min(maxTermIndex, commitIndex[i]))
     Otherwise the prefix is the empty tuple
     ELSE ⟨⟩

CheckIsPrefix(seq1, seq2) ≜
    ∧ Len(seq1) ≤ Len(seq2)
    ∧ ∀ i ∈ 1 .. Len(seq1) : seq1[i] = seq2[i]

The prefix of the log of server i that has been committed
Committed(i) ≜
    IF commitIndex[i] = 0
     THEN ⟨⟩
     ELSE SubSeq(log[i], 1, commitIndex[i])

MyConstraint ≜ (∀ i ∈ Servers : currentTerm[i] ≤ MaxTerm
                  ∧ Len(log[i]) ≤ MaxClientRequests)
                  ∧ (∀ m ∈ DOMAIN messages : messages[m] ≤ 1)
```

$Symmetry \triangleq Permutations(Servers)$

********** *INIT* **********

$InitHistoryVars \triangleq voterLog = [i \in Servers \mapsto [j \in \{\} \mapsto \langle\rangle]]$

$$InitServerVars \triangleq \begin{array}{l} \land\ currentTerm = [i \in Servers \mapsto 1] \\ \land\ state = [i \in Servers \mapsto Follower] \\ \land\ votedFor = [i \in Servers \mapsto Nil] \end{array}$$

$$InitCandidateVars \triangleq \begin{array}{l} \land\ votesResponded = [i \in Servers \mapsto \{\}] \\ \land\ votesGranted = [i \in Servers \mapsto \{\}] \end{array}$$

The values $nextIndex[i][i]$ and $matchIndex[i][i]$ are never read, since the leader does not send itself messages. It's still easier to include these in the functions.

$$InitLeaderVars \triangleq \begin{array}{l} \land\ nextIndex = [i \in Servers \mapsto [j \in Servers \mapsto 1]] \\ \land\ matchIndex = [i \in Servers \mapsto [j \in Servers \mapsto 0]] \end{array}$$

$$InitLogVars \triangleq \begin{array}{l} \land\ log = [i \in Servers \mapsto \langle\rangle] \\ \land\ commitIndex = [i \in Servers \mapsto 0] \end{array}$$

$$\begin{array}{l} InitHovercraftVars \triangleq \\ \quad \land\ switchBuffer = [v \in \{\} \mapsto \langle\rangle] \\ \quad \land\ unorderedRequests = [s \in Server \mapsto \{\}] \\ \quad \land\ switchSentRecord = [s \in Server \mapsto \{\}] \end{array}$$

$$\begin{array}{l} Init \triangleq \land\ messages = [m \in \{\} \mapsto 0] \\ \qquad \land\ InitHistoryVars \\ \qquad \land\ InitServerVars \\ \qquad \land\ InitCandidateVars \\ \qquad \land\ InitLeaderVars \\ \qquad \land\ InitLogVars \\ \qquad \land\ maxc = 0 \\ \qquad \land\ leaderCount = [i \in Servers \mapsto 0] \\ \qquad \land\ entryCommitStats = [idx\_term \in \{\} \mapsto \\ \qquad\qquad [sentCount \mapsto 0, \\ \qquad\qquad\ ackCount \mapsto 0, \\ \qquad\qquad\ committed \mapsto \text{FALSE}]] \\ \qquad \land\ InitHovercraftVars \\ \qquad \land\ switchIndex = 4 \\ \qquad \land\ Servers = Server \setminus \{switchIndex\} \end{array}$$

used to start from a state with a *Leader*
we only verify normal case excluding leader election

$$\begin{array}{l} MyInit \triangleq \\ \quad \text{LET } ServerSet4 \triangleq \text{CHOOSE } S \in \text{SUBSET } (Server) : Cardinality(S) = 4 \\ \qquad TheSwitchId \triangleq \text{CHOOSE } s \in ServerSet4 : \text{TRUE} \end{array}$$

$TheLeaderId \triangleq \text{CHOOSE } l \in ServerSet4 \setminus \{TheSwitchId\} : \text{TRUE}$
$FollowerIds \triangleq ServerSet4 \setminus \{TheSwitchId,\ TheLeaderId\}$

Define state based on these fixed roles

$$
\begin{aligned}
TheState \triangleq\ & [s \in Server \mapsto \\
& \quad \text{IF } s = TheSwitchId \text{ THEN } Switch \\
& \quad \text{ELSE IF } s = TheLeaderId \text{ THEN } Leader \\
& \quad \text{ELSE IF } s \in FollowerIds \text{ THEN } Follower \\
& \quad \text{ELSE } Follower \\
&]
\end{aligned}
$$

$TheSwitchIndex \triangleq TheSwitchId$
$TheServersSet \triangleq Server \setminus \{TheSwitchIndex\}$
$Voters \triangleq TheServersSet \setminus \{TheLeaderId\}$

IN

Constraint: Ensure *Server* has enough elements

$\land\ Cardinality(Server) \geq 4$

Add *PrintT* to verify this version

$\land\ PrintT(\text{“}MyInit : switchIndex = \text{”} \circ ToString(TheSwitchIndex))$
$\land\ PrintT(\text{“}MyInit : state[switchIndex] = \text{”} \circ ToString(TheState[TheSwitchIndex]))$
$\land\ PrintT(\text{“}MyInit : Leader\ is = \text{”} \circ ToString(TheLeaderId))$
$\land\ PrintT(\text{“}MyInit : state[LeaderId] = \text{”} \circ ToString(TheState[TheLeaderId]))$
$\land\ PrintT(\text{“}MyInit : Servers = \text{”} \circ ToString(TheServersSet))$
$\land\ PrintT(\text{“}MyInit : switchBuffer\ Domain = \text{”} \circ ToString(\text{DOMAIN } [v \in \{\} \mapsto \{\}]))$

$\land\ commitIndex = [s \in Server \mapsto 0]$
$\land\ currentTerm = [s \in Server \mapsto 2]$
$\land\ leaderCount = [s \in Server \mapsto \text{IF } s = TheLeaderId \text{ THEN } 1 \text{ ELSE } 0]$
$\land\ log = [s \in Server \mapsto \langle\rangle]$
$\land\ matchIndex = [s \in Server \mapsto [t \in Server \mapsto 0]]$
$\land\ maxc = 0$
$\land\ messages = [m \in \{\} \mapsto 0]$
$\land\ nextIndex = [s \in Server \mapsto [t \in Server \mapsto 1]]$
$\land\ state = TheState$

$$
\begin{aligned}
\land\ votedFor = [s \in Server \mapsto\ & \\
& \text{IF } s = TheLeaderId \text{ THEN } Nil \text{ ELSE } TheLeaderId]
\end{aligned}
$$

$$
\begin{aligned}
\land\ voterLog = [s \in Server \mapsto\ & \\
& \text{IF } s = TheLeaderId \text{ THEN} \\
& [v \in Voters \mapsto \langle\rangle] \text{ ELSE } [v \in \{\} \mapsto \langle\rangle]]
\end{aligned}
$$

$$
\begin{aligned}
\land\ votesGranted = [s \in Server \mapsto\ & \\
& \text{IF } s = TheLeaderId \text{ THEN } Voters \text{ ELSE } \{\}]
\end{aligned}
$$

$$
\begin{aligned}
\land\ votesResponded = [s \in Server \mapsto\ & \\
& \text{IF } s = TheLeaderId \text{ THEN } Voters \text{ ELSE } \{\}]
\end{aligned}
$$

$$
\begin{aligned}
\land\ entryCommitStats = [idx\_term \in \{\} \mapsto\ & \\
& [sentCount \mapsto 0, \\
& \ ackCount \mapsto 0,
\end{aligned}
$$

$committed \mapsto \text{FALSE}]]$

$\land\ switchBuffer = [v\ \in \{\} \mapsto \{\}]$

$\land\ unorderedRequests = [s \in Server \mapsto \{\}]$

$\land\ switchSentRecord = [s \in Server \mapsto \{\}]$

$\land\ switchIndex = TheSwitchIndex$

$\land\ Servers = TheServersSet$

********** Define state transitions**********

Modified to limit Restarts only for Leaders

Server i restarts from stable storage.

It loses everything but its *currentTerm*, *votedFor*, and *log*.

Also persists messages and instrumentation *vars*

$Restart(i) \triangleq$

$\land\ state[i] = Leader$

$\land\ state'\ = [state \text{ EXCEPT } ![i] = Follower]$

$\land\ votesResponded' = [votesResponded \text{ EXCEPT } ![i] = \{\}]$

$\land\ votesGranted' = [votesGranted \text{ EXCEPT } ![i] = \{\}]$

$\land\ voterLog' = [voterLog \text{ EXCEPT } ![i] = [j \in \{\} \mapsto \langle\rangle]]$

$\land\ nextIndex' = [nextIndex \text{ EXCEPT } ![i] = [j \in Server \mapsto 1]]$

$\land\ matchIndex' = [matchIndex \text{ EXCEPT } ![i] = [j \in Server \mapsto 0]]$

$\land\ commitIndex' = [commitIndex \text{ EXCEPT } ![i] = 0]$

$\land\ unorderedRequests' = [unorderedRequests \text{ EXCEPT } ![i] = \{\}]$

$\land\ switchSentRecord' = [switchSentRecord \text{ EXCEPT } ![i] = \{\}]$

$\land\ \text{UNCHANGED } \langle messages, currentTerm, votedFor, log, instrumentationVars, switchIndex, switchBuffer, Servers\rangle$

Modified to restrict *Timeout* to just Followers

Server i times out and starts a new election. *Follower* $\rightarrow$ *Candidate*

$Timeout(i) \triangleq\ \land\ state[i] \in \{Follower\}$, *Candidate*

$\land\ currentTerm[i] < MaxTerm$

$\land\ state' = [state \text{ EXCEPT } ![i] = Candidate]$

$\land\ currentTerm' = [currentTerm \text{ EXCEPT } ![i] = currentTerm[i] + 1]$

Most implementations would probably just set the local vote atomically, but messaging localhost for it is weaker.

$\land\ votedFor' = [votedFor \text{ EXCEPT } ![i] = Nil]$

$\land\ votesResponded' = [votesResponded \text{ EXCEPT } ![i] = \{\}]$

$\land\ votesGranted' = [votesGranted \text{ EXCEPT } ![i] = \{\}]$

$\land\ voterLog' = [voterLog \text{ EXCEPT } ![i] = [j \in \{\} \mapsto \langle\rangle]]$

$\land\ \text{UNCHANGED } \langle messages, leaderVars, logVars, instrumentationVars, hovercraftVars, Servers\rangle$

Modified to restrict *Leader* transitions, bounded by *MaxBecomeLeader*

Candidate i transitions to leader. *Candidate* $\rightarrow$ *Leader*

$BecomeLeader(i) \triangleq$

$\land\ state[i] = Candidate$

$\land\ votesGranted[i] \in Quorum$
$\land\ leaderCount[i] < MaxBecomeLeader$
$\land\ state' = [state \text{ EXCEPT } ![i] = Leader]$
$\land\ nextIndex' = [nextIndex \text{ EXCEPT } ![i] = [j \in Server \mapsto Len(log[i]) + 1]]$
$\land\ matchIndex' = [matchIndex \text{ EXCEPT } ![i] = [j \in Server \mapsto 0]]$
$\land\ leaderCount' = [leaderCount \text{ EXCEPT } ![i] = leaderCount[i] + 1]$
$\land$ UNCHANGED $\langle messages, currentTerm, votedFor, candidateVars, logVars, maxc, entryCommitStats, hovercraftVars, Servers\rangle$

Modified up to *MaxTerm*; Back To *Follower*
Any *RPC* with a newer term causes the recipient to advance its term first.
$UpdateTerm(i, j, m) \triangleq$
$\land\ state[i] \neq Switch \land state[j] \neq Switch$
$\land\ m.mterm > currentTerm[i]$
$\land\ m.mterm < MaxTerm$
$\land\ currentTerm' = [currentTerm \text{ EXCEPT } ![i] = m.mterm]$
$\land\ state' = [state \text{ EXCEPT } ![i] = Follower]$
$\land\ votedFor' = [votedFor \text{ EXCEPT } ![i] = Nil]$
messages is unchanged so m can be processed further.
$\land$ UNCHANGED $\langle messages, candidateVars, leaderVars, logVars, instrumentationVars, hovercraftVars, Servers\rangle$

********** REQUEST VOTE **********

Message handlers
$i =$ recipient, $j =$ sender, $m =$ message

Candidate i sends j a *RequestVote* request.
$RequestVote(i, j) \triangleq$
$\land\ state[i] = Candidate$
$\land\ state[j] \neq Switch$
$\land\ j \notin votesResponded[i]$
$\land\ Send([mtype \mapsto RequestVoteRequest,$
$mterm \mapsto currentTerm[i],$
$mlastLogTerm \mapsto LastTerm(log[i]),$
$mlastLogIndex \mapsto Len(log[i]),$
$msource \mapsto i,$
$mdest \mapsto j])$
$\land$ UNCHANGED $\langle serverVars, candidateVars, leaderVars, logVars, instrumentationVars, hovercraftVars, Servers\rangle$

Server i receives a *RequestVote* request from server j with
$m.mterm \leq currentTerm[i]$.
$HandleRequestVoteRequest(i, j, m) \triangleq$

```
    LET logOk ≜  ∨ m.mlastLogTerm > LastTerm(log[i])
                 ∨ ∧ m.mlastLogTerm = LastTerm(log[i])
                   ∧ m.mlastLogIndex ≥ Len(log[i])
        grant ≜  ∧ m.mterm = currentTerm[i]
                 ∧ logOk
                 ∧ votedFor[i] ∈ {Nil, j}
    IN  ∧ m.mterm ≤ currentTerm[i]
        ∧ ∨ grant  ∧ votedFor' = [votedFor EXCEPT ![i] = j]
          ∨ ¬grant ∧ UNCHANGED votedFor
        ∧ Reply([mtype          ↦ RequestVoteResponse,
                 mterm          ↦ currentTerm[i],
                 mvoteGranted   ↦ grant,
                  mlog is used just for the elections history variable for
                  the proof. It would not exist in a real implementation.
                 mlog           ↦ log[i],
                 msource        ↦ i,
                 mdest          ↦ j],
                 m)
        ∧ UNCHANGED ⟨state, currentTerm, candidateVars, leaderVars, logVars,
                     instrumentationVars, hovercraftVars, Servers⟩

 Server i receives a RequestVote response from server j with
 m.mterm = currentTerm[i].
HandleRequestVoteResponse(i, j, m) ≜
    This tallies votes even when the current state is not Candidate, but
    they won't be looked at, so it doesn't matter.
    ∧ m.mterm = currentTerm[i]
    ∧ votesResponded' = [votesResponded EXCEPT ![i] =
                            votesResponded[i] ∪ {j}]
    ∧ ∨ ∧ m.mvoteGranted
        ∧ votesGranted' = [votesGranted EXCEPT ![i] =
                              votesGranted[i] ∪ {j}]
        ∧ voterLog' = [voterLog EXCEPT ![i] =
                          voterLog[i] @@ (j :> m.mlog)]
      ∨ ∧ ¬m.mvoteGranted
        ∧ UNCHANGED ⟨votesGranted, voterLog⟩
    ∧ Discard(m)
    ∧ UNCHANGED ⟨serverVars, votedFor, leaderVars, logVars,
                 instrumentationVars, hovercraftVars, Servers⟩

 Responses with stale terms are ignored.
DropStaleResponse(i, j, m) ≜
    ∧ m.mterm < currentTerm[i]
    ∧ Discard(m)
    ∧ UNCHANGED ⟨serverVars, candidateVars, leaderVars, logVars,
```

$instrumentationVars,\ hovercraftVars,\ Servers\rangle$

********** *AppendEntries* **********

Old raft. Leader i receives a client request to add v to the *log*.

$ClientRequest(i,\ v)\ \triangleq$
- $\land\ state[i] = Leader$
- $\land\ maxc < MaxClientRequests$
- $\land$ LET $entryTerm\ \triangleq\ currentTerm[i]$
  - $entry\ \triangleq\ [term \mapsto entryTerm,\ value \mapsto v]$
  - $entryExists\ \triangleq\ \exists j \in$ DOMAIN $log[i] :$
    $log[i][j].value = v \land log[i][j].term = entryTerm$
  - $newLog\ \triangleq$ IF $entryExists$ THEN $log[i]$ ELSE $Append(log[i],\ entry)$
  - $newEntryIndex\ \triangleq\ Len(log[i]) + 1$
  - $newEntryKey\ \triangleq\ \langle newEntryIndex,\ entryTerm\rangle$

  IN
  - $\land\ log' = [log$ EXCEPT $![i] = newLog]$
  - $\land\ maxc' =$ IF $entryExists$ THEN $maxc$ ELSE $maxc + 1$
  - $\land\ entryCommitStats' =$
    IF $\neg entryExists \land newEntryIndex > 0$
    THEN $entryCommitStats$ @@ $(newEntryKey :> [sentCount \mapsto 0,$
    $ackCount \mapsto 0,\ committed \mapsto$ FALSE$])$
    ELSE $entryCommitStats$
- $\land$ UNCHANGED $\langle messages,\ serverVars,\ candidateVars,\ leaderVars,$
  $commitIndex,\ leaderCount,\ hovercraftVars,\ Servers\rangle$

Leader i ingests a request v that has been replicated to its unordered set.

$LeaderIngestHovercRaftRequest(i,\ v)\ \triangleq$
- $\land\ state[i] = Leader$
- $\land\ v \in unorderedRequests[i]$ Request *ID* is pending for the leader
- $\land\ v \in$ DOMAIN $switchBuffer$ Full request data $\exists$ in switch buffer
- $\land\ maxc < MaxClientRequests$
- $\land$ LET $entryFromBuffer\ \triangleq\ switchBuffer[v]$

  Use leader's current term, keep value and *payload* from buffer
  - $newEntry\ \triangleq\ [term \mapsto currentTerm[i],$
    $value \mapsto v,$
    $payload \mapsto entryFromBuffer.payload]$
  - $entryExists\ \triangleq\ \exists k \in$ DOMAIN $log[i] :$
    $log[i][k].value = v \land log[i][k].term = newEntry.term$
  - $newLog\ \triangleq$ IF $entryExists$ THEN $log[i]$ ELSE $Append(log[i],\ newEntry)$
  - $newEntryIndex\ \triangleq\ Len(log[i]) + 1$
  - $newEntryKey\ \triangleq\ \langle newEntryIndex,\ newEntry.term\rangle$

  IN

  Debug Print (keep for verification)

$\land\ PrintT(\text{“}LIR : Ingesting\ v = \text{”} \circ ToString(v) \circ \text{“}into\ log\text{”} \circ ToString(i))$

$\land\ PrintT(\text{“}LIR : newLog = \text{”} \circ ToString(newLog))$

$\land\ log' = [log \text{ EXCEPT } ![i] = newLog]$
$\land\ maxc' = \text{IF } entryExists \text{ THEN } maxc \text{ ELSE } maxc + 1$
$\land\ entryCommitStats' =$
IF $\neg entryExists \land newEntryIndex > 0$
THEN $entryCommitStats \mathbin{@@} (newEntryKey :> [sentCount \mapsto 0,$
$ackCount \mapsto 0,\ committed \mapsto \text{FALSE}])$
ELSE $entryCommitStats$
$\land\ unorderedRequests' = [unorderedRequests \text{ EXCEPT } ![i] = @ \setminus \{v\}]$
$\land$ UNCHANGED $\langle messages,\ serverVars,\ candidateVars,\ leaderVars,$
$commitIndex,\ leaderCount,\ switchIndex,\ switchBuffer,$
$Servers,\ switchSentRecord\rangle$

Client sends a request to the *Switch*, which buffers it,
not yet replicated to servers
s is the *Switch*, i is the leader, v is the request value
(along with term will represent a request *ID* in this model)
$SwitchClientRequest(s,\ i,\ v) \triangleq$
$\land\ state[s] = Switch$ Only the switch server can process client requests
$\land\ state[i] = Leader$
$\land\ v \notin \text{DOMAIN } switchBuffer$ Only process new requests
$\land$ LET $entryWithPayload \triangleq [term \mapsto currentTerm[i],$
$value \mapsto v,\ payload \mapsto v]$
IN
$\land\ switchBuffer' = switchBuffer \mathbin{@@} (v :> entryWithPayload)$
$\land\ unorderedRequests' =$
$[unorderedRequests \text{ EXCEPT } ![s] = unorderedRequests[s] \cup \{v\}]$
$\land$ UNCHANGED $\langle messages,\ serverVars,\ candidateVars,\ leaderVars,\ logVars,$
$leaderCount,\ entryCommitStats,\ switchIndex,\ maxc,$
$Servers,\ switchSentRecord\rangle$

The *Switch* replicates v to one server 'i' from *Servers*.
Prevents resending the same $\langle v,\ term\rangle$ pair to the same server 'i'.
$SwitchClientRequestReplicate(s,\ i,\ v) \triangleq$
$\land\ state[s] = Switch$ Only the switch server can replicate requests
$\land\ state[i] \neq Switch$ Target server is not the switch
$\land\ v \in unorderedRequests[s]$ Request must be pending at the switch
$\land$ LET $entryInBuffer \triangleq switchBuffer[v]$
$requestTerm \triangleq entryInBuffer.term$
$requestPair \triangleq \langle v,\ requestTerm\rangle$
IN
Ensure this specific v/term pair hasn't been sent to i yet
$\land\ requestPair \notin switchSentRecord[i]$
Check that the target doesn't already have it pending
$\land\ v \notin unorderedRequests[i]$
$\land\ unorderedRequests' = [unorderedRequests \text{ EXCEPT } ![i] = @ \cup \{v\}]$

$\land\ switchSentRecord' =$
$[switchSentRecord \text{ EXCEPT } ![i] = @ \cup \{requestPair\}]$
$\land$ UNCHANGED $\langle messages,\ serverVars,\ candidateVars,\ leaderVars,\ logVars,$
$leaderCount,\ entryCommitStats,\ switchIndex,\ switchBuffer,$
$maxc,\ Servers\rangle$

Modified. Leader i sends j an *AppendEntries* request containing exactly 1 entry.
While implementations may want to send more than 1 at a time, this spec uses
just 1 because it minimizes atomic regions without loss of generality.
Sending empty entries is done for telling followers *Leader* is alive.

$AppendEntries(i,\ j) \triangleq$
$\land\ i \neq j$
$\land\ state[i] = Leader$
$\land\ Len(log[i]) > 0$
Only proceed if the leader has entries to send
$\land\ nextIndex[i][j] \leq Len(log[i])$
Only proceed if there are entries to send to this follower
$\land\ matchIndex[i][j] < nextIndex[i][j]$
Only send if follower hasn't already acknowledged this index
$\land$ LET $entryIndex \triangleq nextIndex[i][j]$
$entry \triangleq log[i][entryIndex]$
$entryMetadata \triangleq [term \mapsto entry.term,\ value \mapsto entry.value]$
$entries \triangleq \langle entryMetadata \rangle$
$entryKey \triangleq \langle entryIndex,\ entry.term \rangle$
$prevLogIndex \triangleq entryIndex - 1$
$prevLogTerm \triangleq$ IF $prevLogIndex > 0$ THEN
$log[i][prevLogIndex].term$
ELSE
0
Send up to 1 entry, constrained by the end of the *log*.
$lastEntry \triangleq Min(\{Len(log[i]),\ nextIndex[i][j]\})$
$entries \triangleq SubSeq(log[i],\ nextIndex[i][j],\ lastEntry)$

IN $Send([mtype \mapsto AppendEntriesRequest,$
$mterm \mapsto currentTerm[i],$
$mprevLogIndex \mapsto prevLogIndex,$
$mprevLogTerm \mapsto prevLogTerm,$
$mentries \mapsto entries,$
mlog is used as a history variable for the proof.
It would not exist in a real implementation.
$mlog \mapsto log[i],$
$mcommitIndex \mapsto Min(\{commitIndex[i],\ entryIndex\}),$
$msource \mapsto i,$
$mdest \mapsto j])$
$\land\ entryCommitStats' =$

```
                IF entryKey ∈ DOMAIN entryCommitStats
                   ∧ ¬entryCommitStats[entryKey].committed
                THEN [entryCommitStats EXCEPT ![entryKey].sentCount = @ + 1]
                ELSE entryCommitStats
        ∧ UNCHANGED ⟨serverVars, candidateVars, leaderVars, logVars, maxc,
                     leaderCount, hovercraftVars, Servers⟩

Server i receives an AppendEntries request from server j with
m.mterm ≤ currentTerm[i]. This just handles m.entries of length 0 or 1, but
implementations could safely accept more by treating them the same as
multiple independent requests of 1 entry.
HandleAppendEntriesRequest(i, j, m) ≜
    LET logOk ≜ ∨ m.mprevLogIndex = 0
                ∨ ∧ m.mprevLogIndex > 0
                  ∧ m.mprevLogIndex ≤ Len(log[i])
                  ∧ m.mprevLogTerm = log[i][m.mprevLogIndex].term
        rejectHovercraftMismatchCondition ≜
            ∧ m.mentries ≠ ⟨⟩   There must be an entry to check
            ∧ LET entry ≜ m.mentries[1]
                  v ≜ entry.value
                  msgTerm ≜ entry.term
              IN
              ¬( ∧ v ∈ unorderedRequests[i]
                 ∧ v ∈ DOMAIN switchBuffer
                 ∧ switchBuffer[v].term = msgTerm)
    IN  ∧ m.mterm ≤ currentTerm[i]
        ∧ ∨ ∧  reject request
              ∨ m.mterm < currentTerm[i]
              ∨ ∧ m.mterm = currentTerm[i]
                ∧ state[i] = Follower
                ∧ ¬logOk
              ∨ ∧ m.mterm = currentTerm[i]
                ∧ state[i] = Follower
                ∧ rejectHovercraftMismatchCondition
            ∧ Reply([mtype          ↦ AppendEntriesResponse,
                     mterm          ↦ currentTerm[i],
                     msuccess       ↦ FALSE,
                     mmatchIndex    ↦ 0,
                     msource        ↦ i,
                     mdest          ↦ j],
                     m)
            ∧ UNCHANGED ⟨serverVars, logVars, unorderedRequests⟩
          ∨  return to follower state
            ∧ m.mterm = currentTerm[i]
            ∧ state[i] = Candidate
```

```
    ∧ state' = [state EXCEPT ![i] = Follower]
    ∧ UNCHANGED ⟨currentTerm, votedFor, logVars, messages,
                 unorderedRequests⟩
  ∨ accept request
    ∧ m.mterm = currentTerm[i]
    ∧ state[i] = Follower
    ∧ logOk
    ∧ LET index ≜ m.mprevLogIndex + 1
      IN  ∨ already done with request
          ∧ ∨ m.mentries = ⟨⟩
            ∨ ∧ m.mentries ≠ ⟨⟩
              ∧ Len(log[i]) ≥ index
              ∧ log[i][index].term = m.mentries[1].term
            This could make our commitIndex decrease (for
            example if we process an old, duplicated request),
            but that doesn't really affect anything.
          ∧ commitIndex' = [commitIndex EXCEPT ![i] =
                                m.mcommitIndex]
∧ commitIndex' = [commitIndex EXCEPT ![i] =
    IF commitIndex[i] < m.mcommitIndex THEN
      Min({m.mcommitIndex, Len(log[i])})
    ELSE
      commitIndex[i]]
          ∧ Reply([mtype       ↦ AppendEntriesResponse,
                   mterm       ↦ currentTerm[i],
                   msuccess    ↦ TRUE,
                   mmatchIndex ↦ m.mprevLogIndex +
                                 Len(m.mentries),
                   msource     ↦ i,
                   mdest       ↦ j],
                  m)
          ∧ UNCHANGED ⟨serverVars, log, unorderedRequests⟩
        ∨ conflict: remove 1 entry
(simplified from original spec - assumes entry length 1)
          ATTENTION since we do not send empty entries,
we have to provide a larger set of Values to ensure progress
          ∧ m.mentries ≠ ⟨⟩
          ∧ Len(log[i]) ≥ index
          ∧ log[i][index].term ≠ m.mentries[1].term
          ∧ LET newLog ≜ SubSeq(log[i], 1, index − 1)
            IN  log' = [log EXCEPT ![i] = newLog]  Truncate log
          ∧ UNCHANGED ⟨serverVars, commitIndex, messages,
                       unorderedRequests⟩
        ∨ no conflict: append entry
          ∧ m.mentries ≠ ⟨⟩
```

```
                    ∧ Len(log[i]) = m.mprevLogIndex
                    ∧ ¬rejectHovercraftMismatchCondition
                    ∧ LET  mark unorderedRequests done
                        entryMetadata ≜ m.mentries[1]
                        fullEntryFromCache ≜ switchBuffer[entryMetadata.value]
                        entryForLocalLog ≜ [term  ↦ entryMetadata.term,
                                            value ↦ entryMetadata.value,
                                            payload ↦ fullEntryFromCache.payload]
                      IN
                        ∧ log' = [log EXCEPT ![i] =
                            Append(log[i], entryForLocalLog)]
                        ∧ unorderedRequests' =
                            [unorderedRequests EXCEPT ![i] =
                            @ \ {entryMetadata.value}]
                    ∧ UNCHANGED ⟨serverVars, commitIndex, messages⟩
          ∧ UNCHANGED ⟨candidateVars, leaderVars, instrumentationVars,
              switchBuffer, switchIndex, switchSentRecord, Servers⟩

Server i receives an AppendEntries response from server j with
m.mterm = currentTerm[i].
HandleAppendEntriesResponse(i, j, m) ≜
    ∧ m.mterm = currentTerm[i]
    ∧ ∨ ∧ m.msuccess  successful
        ∧ LET
              newMatchIndex ≜ m.mmatchIndex
              entryKey ≜ IF newMatchIndex > 0 ∧ newMatchIndex ≤ Len(log[i])
                         THEN ⟨newMatchIndex, log[i][newMatchIndex].term⟩
                         ELSE ⟨0, 0⟩  Invalid index or empty log
          IN  ∧ nextIndex'  = [nextIndex  EXCEPT ![i][j]  = m.mmatchIndex + 1]
              ∧ matchIndex' = [matchIndex EXCEPT ![i][j] = m.mmatchIndex]
              ∧ entryCommitStats' =
                  IF  ∧ entryKey ≠ ⟨0, 0⟩
                      ∧ entryKey ∈ DOMAIN entryCommitStats
                      ∧ ¬entryCommitStats[entryKey].committed
                  THEN [entryCommitStats EXCEPT ![entryKey].ackCount = @ + 1]
                  ELSE entryCommitStats
      ∨ ∧ ¬m.msuccess  not successful
        ∧ nextIndex' = [nextIndex EXCEPT ![i][j] =
                            Max({nextIndex[i][j] − 1, 1})]
        ∧ UNCHANGED ⟨matchIndex, entryCommitStats⟩
    ∧ Discard(m)
    ∧ UNCHANGED ⟨serverVars, candidateVars, logVars, maxc, leaderCount, hovercraftVars, Servers⟩

Leader i advances its commitIndex.
This is done as a separate step from handling AppendEntries responses,
```

```
in part to minimize atomic regions, and in part so that leaders of
single-server clusters are able to mark entries committed.
AdvanceCommitIndex(i) ≜
    ∧ state[i] = Leader
    ∧ LET  The set of servers that agree up through index.
           Agree(index) ≜ {i} ∪ {k ∈ Server :
                                 matchIndex[i][k] ≥ index}
           The maximum indexes for which a quorum agrees
           agreeIndexes ≜ {index ∈ 1 .. Len(log[i]) :
                             Agree(index) ∈ Quorum}
           New value for commitIndex'[i]
           newCommitIndex ≜
              IF ∧ agreeIndexes ≠ {}
                 ∧ log[i][Max(agreeIndexes)].term = currentTerm[i]
              THEN
                 Max(agreeIndexes)
              ELSE
                 commitIndex[i]
           committedIndexes ≜ {k ∈ Nat : ∧ k > commitIndex[i]
                                         ∧ k ≤ newCommitIndex}
           Identify the keys in entryCommitStats
           corresponding to newly committed entries
           keysToUpdate ≜ {key ∈ DOMAIN entryCommitStats :
                           key[1] ∈ committedIndexes}
       IN  ∧ commitIndex' = [commitIndex EXCEPT ![i] = newCommitIndex]
           ∧ entryCommitStats' =
               [key ∈ DOMAIN entryCommitStats ↦
                  IF key ∈ keysToUpdate
                  THEN [entryCommitStats[key] EXCEPT !.committed = TRUE]
                  ELSE entryCommitStats[key]]
    ∧ UNCHANGED ⟨messages, serverVars, candidateVars, leaderVars, log, maxc,
                 leaderCount, hovercraftVars, Servers⟩

Network state transitions

The network duplicates a message
DuplicateMessage(m) ≜
    ∧ Send(m)
    ∧ UNCHANGED ⟨serverVars, candidateVars, leaderVars, logVars,
                 instrumentationVars, hovercraftVars, Servers⟩

The network drops a message
DropMessage(m) ≜
    ∧ Discard(m)
    ∧ UNCHANGED ⟨serverVars, candidateVars, leaderVars, logVars,
                 instrumentationVars, hovercraftVars, Servers⟩
```

```
 Receive a message.
Receive(m) ≜
    LET i ≜ m.mdest
        j ≜ m.msource
    IN  Any RPC with a newer term causes the recipient to advance
        its term first. Responses with stale terms are ignored.
        ∨ UpdateTerm(i, j, m)
        ∨ ∧ m.mtype = RequestVoteRequest
          ∧ HandleRequestVoteRequest(i, j, m)
        ∨ ∧ m.mtype = RequestVoteResponse
          ∧ ∨ DropStaleResponse(i, j, m)
            ∨ HandleRequestVoteResponse(i, j, m)
        ∨ ∧ m.mtype = AppendEntriesRequest
          ∧ HandleAppendEntriesRequest(i, j, m)
        ∨ ∧ m.mtype = AppendEntriesResponse
          ∧ ∨ DropStaleResponse(i, j, m)
            ∨ HandleAppendEntriesResponse(i, j, m)

 Defines how the variables may transition.
Next ≜
        ∨ ∃ i ∈ Server : Timeout(i)
        ∨ ∃ i ∈ Server : Restart(i)
        ∨ ∃ i, j ∈ Server : i ≠ j ∧ RequestVote(i, j)
        ∨ ∃ i ∈ Server : BecomeLeader(i)
        ∨ ∃ i ∈ Server, v ∈ Value :
          state[i] = Leader ∧ ClientRequest(i, v)
        ∨ ∃ i ∈ Server : AdvanceCommitIndex(i)
        ∨ ∃ i, j ∈ Server : i ≠ j ∧ AppendEntries(i, j)
        ∨ ∃ m ∈ {msg ∈ ValidMessage(messages) :
                 msg.mtype ∈ {RequestVoteRequest, RequestVoteResponse,
                 AppendEntriesRequest, AppendEntriesResponse}} : Receive(m)
        ∨ ∃ m ∈ {msg ∈ ValidMessage(messages) :
                 msg.mtype ∈ {AppendEntriesRequest}} : DuplicateMessage(m)
        ∨ ∃ m ∈ {msg ∈ ValidMessage(messages) :
                 msg.mtype ∈ {RequestVoteRequest}} : DropMessage(m)

MyNext ≜
        ∨ ∃ i ∈ Server, v ∈ Value :
          state[i] = Leader ∧ ClientRequest(i, v)
        ∨ ∃ i ∈ Server : AdvanceCommitIndex(i)
        ∨ ∃ i, j ∈ Server : i ≠ j ∧ AppendEntries(i, j)
        ∨ ∃ m ∈ {msg ∈ ValidMessage(messages) : msg.mtype ∈
        {AppendEntriesRequest, AppendEntriesResponse}} : Receive(m)

MySwitchNext ≜
```

$\lor \exists\, i \in Servers,\, v \in Value :$
$state[i] = Leader \land SwitchClientRequest(switchIndex,\, i,\, v)$
$\lor \exists\, i \in Servers,\, v \in \text{DOMAIN } switchBuffer :$
$SwitchClientRequestReplicate(switchIndex,\, i,\, v)$
$\lor \exists\, i \in Servers,\, v \in \text{DOMAIN } switchBuffer :$
$state[i] = Leader \land LeaderIngestHovercRaftRequest(i,\, v)$
$\lor \exists\, i \in Servers : AdvanceCommitIndex(i)$
$\lor \exists\, i,\, j \in Servers : i \neq j \land AppendEntries(i,\, j)$
$\lor \exists\, m \in \{msg \in ValidMessage(messages) : msg.mtype \in$
$\{AppendEntriesRequest,\, AppendEntriesResponse\}\} : Receive(m)$

The specification must start with the initial state and transition according to *Next*.

$Spec \triangleq Init \land \Box[Next]\_vars$

$MySpec \triangleq MyInit \land \Box[MyNext]\_vars$

$MySwitchSpec \triangleq MyInit \land \Box[MySwitchNext]_{vars}$

Invariants

$MoreThanOneLeaderInv \triangleq$
$\forall\, i,\, j \in Server :$
$(\land currentTerm[i] = currentTerm[j]$
$\land state[i] = Leader$
$\land state[j] = Leader)$
$\Rightarrow i = j$

Every (index, term) pair determines a *log* prefix.
From page 8 of the Raft paper: "If two logs contain an entry with the same index and term, then the logs are identical in all preceding entries."

$LogMatchingInv \triangleq$
$\forall\, i,\, j \in Server : i \neq j \Rightarrow$
$\forall\, n \in 1 \,..\, min(Len(log[i]),\, Len(log[j])) :$
$log[i][n].term = log[j][n].term \Rightarrow$
$SubSeq(log[i],\, 1,\, n) = SubSeq(log[j],\, 1,\, n)$

The committed entries in every *log* are a prefix of the leader's *log* up to the leader's term (since a next *Leader* may already be elected without the old leader stepping down yet)

$LeaderCompletenessInv \triangleq$
$\forall\, i \in Server :$
$state[i] = Leader \Rightarrow$
$\forall\, j \in Server : i \neq j \Rightarrow$
$CheckIsPrefix(CommittedTermPrefix(j,\, currentTerm[i]),\, log[i])$

```
 Committed log entries should never conflict between servers
LogInv ≜
    ∀ i, j ∈ Server :
        ∨ CheckIsPrefix(Committed(i), Committed(j))
        ∨ CheckIsPrefix(Committed(j), Committed(i))

 Note that LogInv checks for safety violations across space
 This is a key safety invariant and should always be checked
THEOREM MySwitchSpec ⇒ (□LogInv ∧ □LeaderCompletenessInv
                                 ∧ □LogMatchingInv ∧ □MoreThanOneLeaderInv)

 instrumentation and performance invariants

 Fake invariant: Checks if every server (including the Switch)
 has exactly one unordered request pending.
AllServersHaveOneUnorderedRequestInv ≜
    ∃ s ∈ Servers :  Cardinality(unorderedRequests[s]) ≠ 2

 A leader's maxc should remain under MaxClientRequests
MaxCInv ≜ (∃ i ∈ Server : state[i] = Leader) ⇒ maxc ≤ MaxClientRequests

 No server can become leader more than MaxBecomeLeader times
LeaderCountInv ≜ ∃ i ∈ Server :
  (state[i] = Leader ⇒ leaderCount[i] ≤ MaxBecomeLeader)

 No server can have a term exceeding MaxTerm
MaxTermInv ≜ ∀ i ∈ Server : currentTerm[i] ≤ MaxTerm

 Check lower bound for message counts on committed entries
 For any entry that has been marked as committed,
 verify that either the number of AppendEntries requests sent OR
 the number of successful acknowledgments received
 is at least the minimum number of followers required to form a majority.
 will fail when an entry was sent twice to a follower
 and no response was acked yet, which is normal
EntryCommitMessageCountInv ≜
    LET NumFollowers ≜ Cardinality(Servers) − 1
        MinFollowersForMajority ≜ Cardinality(Servers) ÷ 2
    IN  ∀ key ∈ DOMAIN entryCommitStats :
          LET stats ≜ entryCommitStats[key]
          IN  IF stats.committed
              THEN (stats.sentCount ≥
                        MinFollowersForMajority ∧ stats.sentCount ≤ NumFollowers)
                        ∨ (stats.ackCount ≥
                        MinFollowersForMajority ∧ stats.ackCount ≤ NumFollowers)
              ELSE  TRUE
```

Check that committed entries received acknowledgments
from a majority of followers.

$EntryCommitAckQuorumInv \triangleq$
  LET $NumServers \triangleq Cardinality(Servers)$
      Minimum number of followers needed (in addition to the leader)
      to reach a majority for committing an entry.
      $MinFollowerAcksForMajority \triangleq NumServers \div 2$
  IN $\forall key \in$ DOMAIN $entryCommitStats$:
      LET $stats \triangleq entryCommitStats[key]$
      IN $stats.committed \Rightarrow (stats.ackCount \geq MinFollowerAcksForMajority)$

fake inv to obtain a trace

$LeaderCommitted \triangleq$
  $\exists\, i \in Servers : commitIndex[i] \neq 1$

\ * Created by Ovidiu-Cristian *Marcu*

With fake Invariant *LeaderCommitted* violated we obtain a trace. Model: $MaxTerm = 3$, $MaxClientRequests = 2$, $Value = \{$"$v1$", "$v2$"$\}$, $Server = \{$"$r1$", "$r2$", "$r3$", "$r4$"$\}$

⟨
[
_*TEAction* ↦ [
  position ↦ 1,
  name ↦ "*Initial predicate*",
  location ↦ "*Unknown location*"
],
Servers ↦ {"$r2$", "$r3$", "$r4$"},
commitIndex ↦ [$r1$ ↦ 0, $r2$ ↦ 0, $r3$ ↦ 0, $r4$ ↦ 0],
currentTerm ↦ [$r1$ ↦ 2, $r2$ ↦ 2, $r3$ ↦ 2, $r4$ ↦ 2],
entryCommitStats ↦ ⟨⟩,
leaderCount ↦ [$r1$ ↦ 0, $r2$ ↦ 1, $r3$ ↦ 0, $r4$ ↦ 0],
log ↦ [$r1$ ↦ ⟨⟩, $r2$ ↦ ⟨⟩, $r3$ ↦ ⟨⟩, $r4$ ↦ ⟨⟩],
matchIndex ↦ [$r1$ ↦ [$r1$ ↦ 0, $r2$ ↦ 0, $r3$ ↦ 0, $r4$ ↦ 0],
 $r2$ ↦ [$r1$ ↦ 0, $r2$ ↦ 0, $r3$ ↦ 0, $r4$ ↦ 0],
 $r3$ ↦ [$r1$ ↦ 0, $r2$ ↦ 0, $r3$ ↦ 0, $r4$ ↦ 0],
 $r4$ ↦ [$r1$ ↦ 0, $r2$ ↦ 0, $r3$ ↦ 0, $r4$ ↦ 0]],
maxc ↦ 0,
messages ↦ ⟨⟩,
nextIndex ↦ [$r1$ ↦ [$r1$ ↦ 1, $r2$ ↦ 1, $r3$ ↦ 1, $r4$ ↦ 1],
 $r2$ ↦ [$r1$ ↦ 1, $r2$ ↦ 1, $r3$ ↦ 1, $r4$ ↦ 1],
 $r3$ ↦ [$r1$ ↦ 1, $r2$ ↦ 1, $r3$ ↦ 1, $r4$ ↦ 1],
 $r4$ ↦ [$r1$ ↦ 1, $r2$ ↦ 1, $r3$ ↦ 1, $r4$ ↦ 1]],
state ↦ [$r1$ ↦ *Switch*, $r2$ ↦ *Leader*, $r3$ ↦ *Follower*, $r4$ ↦ *Follower*],
switchBuffer ↦ ⟨⟩,
switchIndex ↦ "$r1$",
switchSentRecord ↦ [$r1$ ↦ {}, $r2$ ↦ {}, $r3$ ↦ {}, $r4$ ↦ {}],
unorderedRequests ↦ [$r1$ ↦ {}, $r2$ ↦ {}, $r3$ ↦ {}, $r4$ ↦ {}],
votedFor ↦ [$r1$ ↦ "$r2$", $r2$ ↦ *Nil*, $r3$ ↦ "$r2$", $r4$ ↦ "$r2$"],

voterLog ↦ [*r*1 ↦ ⟨⟩, *r*2 ↦ [*r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩], *r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩],
votesGranted ↦ [*r*1 ↦ {}, *r*2 ↦ {"*r*3", "*r*4"}, *r*3 ↦ {}, *r*4 ↦ {}],
votesResponded ↦ [*r*1 ↦ {}, *r*2 ↦ {"*r*3", "*r*4"}, *r*3 ↦ {}, *r*4 ↦ {}]
],
[
_*TEAction* ↦ [
position ↦ 2,
name ↦ "*MySwitchNext*",
location ↦ "*line* 842, *col* 7 *to line* 843, *col* 63 *of module hovercraft*"
],
Servers ↦ {"*r*2", "*r*3", "*r*4"},
commitIndex ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
currentTerm ↦ [*r*1 ↦ 2, *r*2 ↦ 2, *r*3 ↦ 2, *r*4 ↦ 2],
entryCommitStats ↦ ⟨⟩,
leaderCount ↦ [*r*1 ↦ 0, *r*2 ↦ 1, *r*3 ↦ 0, *r*4 ↦ 0],
log ↦ [*r*1 ↦ ⟨⟩, *r*2 ↦ ⟨⟩, *r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩],
matchIndex ↦ [*r*1 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
*r*2 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
*r*3 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
*r*4 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0]],
maxc ↦ 0,
messages ↦ ⟨⟩,
nextIndex ↦ [*r*1 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1],
*r*2 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1],
*r*3 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1],
*r*4 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1]],
state ↦ [*r*1 ↦ *Switch*, *r*2 ↦ *Leader*, *r*3 ↦ *Follower*, *r*4 ↦ *Follower*],
switchBuffer ↦ [*v*1 ↦ [*term* ↦ 2, *value* ↦ "*v*1", *payload* ↦ "*v*1"]],
switchIndex ↦ "*r*1",
switchSentRecord ↦ [*r*1 ↦ {}, *r*2 ↦ {}, *r*3 ↦ {}, *r*4 ↦ {}],
unorderedRequests ↦ [*r*1 ↦ {"*v*1"}, *r*2 ↦ {}, *r*3 ↦ {}, *r*4 ↦ {}],
votedFor ↦ [*r*1 ↦ "*r*2", *r*2 ↦ *Nil*, *r*3 ↦ "*r*2", *r*4 ↦ "*r*2"],
voterLog ↦ [*r*1 ↦ ⟨⟩, *r*2 ↦ [*r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩], *r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩],
votesGranted ↦ [*r*1 ↦ {}, *r*2 ↦ {"*r*3", "*r*4"}, *r*3 ↦ {}, *r*4 ↦ {}],
votesResponded ↦ [*r*1 ↦ {}, *r*2 ↦ {"*r*3", "*r*4"}, *r*3 ↦ {}, *r*4 ↦ {}]
],
[
_*TEAction* ↦ [
position ↦ 3,
name ↦ "*MySwitchNext*",
location ↦ "*line* 842, *col* 7 *to line* 843, *col* 63 *of module hovercraft*"
],
Servers ↦ {"*r*2", "*r*3", "*r*4"},
commitIndex ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
currentTerm ↦ [*r*1 ↦ 2, *r*2 ↦ 2, *r*3 ↦ 2, *r*4 ↦ 2],
entryCommitStats ↦ ⟨⟩,
leaderCount ↦ [*r*1 ↦ 0, *r*2 ↦ 1, *r*3 ↦ 0, *r*4 ↦ 0],
log ↦ [*r*1 ↦ ⟨⟩, *r*2 ↦ ⟨⟩, *r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩],
matchIndex ↦ [*r*1 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
*r*2 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],

$r3 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$r4 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]]$,
$maxc \mapsto 0$,
$messages \mapsto \langle\rangle$,
$nextIndex \mapsto [r1 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r2 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r3 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r4 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]]$,
$state \mapsto [r1 \mapsto Switch,\ r2 \mapsto Leader,\ r3 \mapsto Follower,\ r4 \mapsto Follower]$,
$switchBuffer \mapsto [v1 \mapsto [term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$v2 \mapsto [term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]],
$switchIndex \mapsto$ "$r1$",
$switchSentRecord \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$,
$unorderedRequests \mapsto [r1 \mapsto \{$"$v1$", "$v2$"$\},\ r2 \mapsto \{\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$,
$votedFor \mapsto [r1 \mapsto$ "$r2$", $r2 \mapsto Nil,\ r3 \mapsto$ "$r2$", $r4 \mapsto$ "$r2$"],
$voterLog \mapsto [r1 \mapsto \langle\rangle,\ r2 \mapsto [r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle],\ r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle]$,
$votesGranted \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{$"$r3$", "$r4$"$\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$,
$votesResponded \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{$"$r3$", "$r4$"$\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$
],
[
$\_TEAction \mapsto$ [
$position \mapsto 4$,
$name \mapsto$ "$MySwitchNext$",
$location \mapsto$ "$line$ 844, col 7 $to\ line$ 845, col 51 $of\ module\ hovercraft$"
],
$Servers \mapsto \{$"$r2$", "$r3$", "$r4$"$\}$,
$commitIndex \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$currentTerm \mapsto [r1 \mapsto 2,\ r2 \mapsto 2,\ r3 \mapsto 2,\ r4 \mapsto 2]$,
$entryCommitStats \mapsto \langle\rangle$,
$leaderCount \mapsto [r1 \mapsto 0,\ r2 \mapsto 1,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$log \mapsto [r1 \mapsto \langle\rangle,\ r2 \mapsto \langle\rangle,\ r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle]$,
$matchIndex \mapsto [r1 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$r2 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$r3 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$r4 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]]$,
$maxc \mapsto 0$,
$messages \mapsto \langle\rangle$,
$nextIndex \mapsto [r1 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r2 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r3 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r4 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]]$,
$state \mapsto [r1 \mapsto Switch,\ r2 \mapsto Leader,\ r3 \mapsto Follower,\ r4 \mapsto Follower]$,
$switchBuffer \mapsto [v1 \mapsto [term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$v2 \mapsto [term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]],
$switchIndex \mapsto$ "$r1$",
$switchSentRecord \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{\langle$"$v1$", $2\rangle\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$,
$unorderedRequests \mapsto [r1 \mapsto \{$"$v1$", "$v2$"$\},\ r2 \mapsto \{$"$v1$"$\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$,
$votedFor \mapsto [r1 \mapsto$ "$r2$", $r2 \mapsto Nil,\ r3 \mapsto$ "$r2$", $r4 \mapsto$ "$r2$"],
$voterLog \mapsto [r1 \mapsto \langle\rangle,\ r2 \mapsto [r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle],\ r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle]$,
$votesGranted \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{$"$r3$", "$r4$"$\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$,

votesResponded ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}]
],
[
_TEAction ↦ [
position ↦ 5,
name ↦ "MySwitchNext",
location ↦ "line 844, col 7 to line 845, col 51 of module hovercraft"
],
Servers ↦ {"r2", "r3", "r4"},
commitIndex ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
currentTerm ↦ [r1 ↦ 2, r2 ↦ 2, r3 ↦ 2, r4 ↦ 2],
entryCommitStats ↦ ⟨⟩,
leaderCount ↦ [r1 ↦ 0, r2 ↦ 1, r3 ↦ 0, r4 ↦ 0],
log ↦ [r1 ↦ ⟨⟩, r2 ↦ ⟨⟩, r3 ↦ ⟨⟩, r4 ↦ ⟨⟩],
matchIndex ↦ [r1 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
r2 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
r3 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
r4 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0]],
maxc ↦ 0,
messages ↦ ⟨⟩,
nextIndex ↦ [r1 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
r2 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
r3 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
r4 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1]],
state ↦ [r1 ↦ Switch, r2 ↦ Leader, r3 ↦ Follower, r4 ↦ Follower],
switchBuffer ↦ [v1 ↦ [term ↦ 2, value ↦ "v1", payload ↦ "v1"],
v2 ↦ [term ↦ 2, value ↦ "v2", payload ↦ "v2"]],
switchIndex ↦ "r1",
switchSentRecord ↦ [r1 ↦ {}, r2 ↦ {⟨"v1", 2⟩}, r3 ↦ {⟨"v1", 2⟩}, r4 ↦ {}],
unorderedRequests ↦ [r1 ↦ {"v1", "v2"}, r2 ↦ {"v1"}, r3 ↦ {"v1"}, r4 ↦ {}],
votedFor ↦ [r1 ↦ "r2", r2 ↦ Nil, r3 ↦ "r2", r4 ↦ "r2"],
voterLog ↦ [r1 ↦ ⟨⟩, r2 ↦ [r3 ↦ ⟨⟩, r4 ↦ ⟨⟩], r3 ↦ ⟨⟩, r4 ↦ ⟨⟩],
votesGranted ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}],
votesResponded ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}]
],
[
_TEAction ↦ [
position ↦ 6,
name ↦ "MySwitchNext",
location ↦ "line 844, col 7 to line 845, col 51 of module hovercraft"
],
Servers ↦ {"r2", "r3", "r4"},
commitIndex ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
currentTerm ↦ [r1 ↦ 2, r2 ↦ 2, r3 ↦ 2, r4 ↦ 2],
entryCommitStats ↦ ⟨⟩,
leaderCount ↦ [r1 ↦ 0, r2 ↦ 1, r3 ↦ 0, r4 ↦ 0],
log ↦ [r1 ↦ ⟨⟩, r2 ↦ ⟨⟩, r3 ↦ ⟨⟩, r4 ↦ ⟨⟩],
matchIndex ↦ [r1 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
r2 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
r3 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],

$r4 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]]$,
$maxc \mapsto 0$,
$messages \mapsto \langle\rangle$,
$nextIndex \mapsto [r1 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r2 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r3 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r4 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]]$,
$state \mapsto [r1 \mapsto Switch,\ r2 \mapsto Leader,\ r3 \mapsto Follower,\ r4 \mapsto Follower]$,
$switchBuffer \mapsto [v1 \mapsto [term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}]$,
$v2 \mapsto [term \mapsto 2,\ value \mapsto \text{“}v2\text{”},\ payload \mapsto \text{“}v2\text{”}]]$,
$switchIndex \mapsto \text{“}r1\text{”}$,
$switchSentRecord \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{\langle \text{“}v1\text{”},\ 2\rangle\},\ r3 \mapsto \{\langle \text{“}v1\text{”},\ 2\rangle\},\ r4 \mapsto \{\langle \text{“}v1\text{”},\ 2\rangle\}]$,
$unorderedRequests \mapsto [r1 \mapsto \{\text{“}v1\text{”},\ \text{“}v2\text{”}\},\ r2 \mapsto \{\text{“}v1\text{”}\},\ r3 \mapsto \{\text{“}v1\text{”}\},\ r4 \mapsto \{\text{“}v1\text{”}\}]$,
$votedFor \mapsto [r1 \mapsto \text{“}r2\text{”},\ r2 \mapsto Nil,\ r3 \mapsto \text{“}r2\text{”},\ r4 \mapsto \text{“}r2\text{”}]$,
$voterLog \mapsto [r1 \mapsto \langle\rangle,\ r2 \mapsto [r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle],\ r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle]$,
$votesGranted \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{\text{“}r3\text{”},\ \text{“}r4\text{”}\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$,
$votesResponded \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{\text{“}r3\text{”},\ \text{“}r4\text{”}\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$
],
[
$\_TEAction \mapsto [$
$position \mapsto 7$,
$name \mapsto \text{“}MySwitchNext\text{”}$,
$location \mapsto \text{“}line\ 844,\ col\ 7\ to\ line\ 845,\ col\ 51\ of\ module\ hovercraft\text{”}$
],
$Servers \mapsto \{\text{“}r2\text{”},\ \text{“}r3\text{”},\ \text{“}r4\text{”}\}$,
$commitIndex \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$currentTerm \mapsto [r1 \mapsto 2,\ r2 \mapsto 2,\ r3 \mapsto 2,\ r4 \mapsto 2]$,
$entryCommitStats \mapsto \langle\rangle$,
$leaderCount \mapsto [r1 \mapsto 0,\ r2 \mapsto 1,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$log \mapsto [r1 \mapsto \langle\rangle,\ r2 \mapsto \langle\rangle,\ r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle]$,
$matchIndex \mapsto [r1 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$r2 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$r3 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$r4 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]]$,
$maxc \mapsto 0$,
$messages \mapsto \langle\rangle$,
$nextIndex \mapsto [r1 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r2 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r3 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r4 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]]$,
$state \mapsto [r1 \mapsto Switch,\ r2 \mapsto Leader,\ r3 \mapsto Follower,\ r4 \mapsto Follower]$,
$switchBuffer \mapsto [v1 \mapsto [term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}]$,
$v2 \mapsto [term \mapsto 2,\ value \mapsto \text{“}v2\text{”},\ payload \mapsto \text{“}v2\text{”}]]$,
$switchIndex \mapsto \text{“}r1\text{”}$,
$switchSentRecord \mapsto [r1 \mapsto \{\}$,
$r2 \mapsto \{\langle \text{“}v1\text{”},\ 2\rangle,\ \langle \text{“}v2\text{”},\ 2\rangle\}$,
$r3 \mapsto \{\langle \text{“}v1\text{”},\ 2\rangle\}$,
$r4 \mapsto \{\langle \text{“}v1\text{”},\ 2\rangle\}]$,
$unorderedRequests \mapsto [r1 \mapsto \{\text{“}v1\text{”},\ \text{“}v2\text{”}\},\ r2 \mapsto \{\text{“}v1\text{”},\ \text{“}v2\text{”}\},\ r3 \mapsto \{\text{“}v1\text{”}\},\ r4 \mapsto \{\text{“}v1\text{”}\}]$,
$votedFor \mapsto [r1 \mapsto \text{“}r2\text{”},\ r2 \mapsto Nil,\ r3 \mapsto \text{“}r2\text{”},\ r4 \mapsto \text{“}r2\text{”}]$,

voterLog ↦ [*r*1 ↦ ⟨⟩, *r*2 ↦ [*r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩], *r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩],
votesGranted ↦ [*r*1 ↦ {}, *r*2 ↦ {"*r*3", "*r*4"}, *r*3 ↦ {}, *r*4 ↦ {}],
votesResponded ↦ [*r*1 ↦ {}, *r*2 ↦ {"*r*3", "*r*4"}, *r*3 ↦ {}, *r*4 ↦ {}]
],
[
_*TEAction* ↦ [
position ↦ 8,
name ↦ "*MySwitchNext*",
location ↦ "*line* 844, *col* 7 *to line* 845, *col* 51 *of module hovercraft*"
],
Servers ↦ {"*r*2", "*r*3", "*r*4"},
commitIndex ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
currentTerm ↦ [*r*1 ↦ 2, *r*2 ↦ 2, *r*3 ↦ 2, *r*4 ↦ 2],
entryCommitStats ↦ ⟨⟩,
leaderCount ↦ [*r*1 ↦ 0, *r*2 ↦ 1, *r*3 ↦ 0, *r*4 ↦ 0],
log ↦ [*r*1 ↦ ⟨⟩, *r*2 ↦ ⟨⟩, *r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩],
matchIndex ↦ [*r*1 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
*r*2 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
*r*3 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
*r*4 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0]],
maxc ↦ 0,
messages ↦ ⟨⟩,
nextIndex ↦ [*r*1 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1],
*r*2 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1],
*r*3 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1],
*r*4 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1]],
state ↦ [*r*1 ↦ *Switch*, *r*2 ↦ *Leader*, *r*3 ↦ *Follower*, *r*4 ↦ *Follower*],
switchBuffer ↦ [*v*1 ↦ [*term* ↦ 2, *value* ↦ "*v*1", *payload* ↦ "*v*1"],
*v*2 ↦ [*term* ↦ 2, *value* ↦ "*v*2", *payload* ↦ "*v*2"]],
switchIndex ↦ "*r*1",
switchSentRecord ↦ [*r*1 ↦ {},
*r*2 ↦ {⟨"*v*1", 2⟩, ⟨"*v*2", 2⟩},
*r*3 ↦ {⟨"*v*1", 2⟩, ⟨"*v*2", 2⟩},
*r*4 ↦ {⟨"*v*1", 2⟩}],
unorderedRequests ↦ [*r*1 ↦ {"*v*1", "*v*2"}, *r*2 ↦ {"*v*1", "*v*2"}, *r*3 ↦ {"*v*1", "*v*2"}, *r*4 ↦ {"*v*1"}],
votedFor ↦ [*r*1 ↦ "*r*2", *r*2 ↦ *Nil*, *r*3 ↦ "*r*2", *r*4 ↦ "*r*2"],
voterLog ↦ [*r*1 ↦ ⟨⟩, *r*2 ↦ [*r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩], *r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩],
votesGranted ↦ [*r*1 ↦ {}, *r*2 ↦ {"*r*3", "*r*4"}, *r*3 ↦ {}, *r*4 ↦ {}],
votesResponded ↦ [*r*1 ↦ {}, *r*2 ↦ {"*r*3", "*r*4"}, *r*3 ↦ {}, *r*4 ↦ {}]
],
[
_*TEAction* ↦ [
position ↦ 9,
name ↦ "*MySwitchNext*",
location ↦ "*line* 846, *col* 7 *to line* 847, *col* 60 *of module hovercraft*"
],
Servers ↦ {"*r*2", "*r*3", "*r*4"},
commitIndex ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
currentTerm ↦ [*r*1 ↦ 2, *r*2 ↦ 2, *r*3 ↦ 2, *r*4 ↦ 2],

entryCommitStats ↦ (⟨1, 2⟩ :> [*sentCount* ↦ 0, *ackCount* ↦ 0, *committed* ↦ FALSE]),
leaderCount ↦ [*r*1 ↦ 0, *r*2 ↦ 1, *r*3 ↦ 0, *r*4 ↦ 0],
log ↦ [*r*1 ↦ ⟨⟩,
 *r*2 ↦ ⟨[*term* ↦ 2, *value* ↦ "*v*1", *payload* ↦ "*v*1"]⟩,
 *r*3 ↦ ⟨⟩,
 *r*4 ↦ ⟨⟩],
matchIndex ↦ [*r*1 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
 *r*2 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
 *r*3 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
 *r*4 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0]],
maxc ↦ 1,
messages ↦ ⟨⟩,
nextIndex ↦ [*r*1 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1],
 *r*2 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1],
 *r*3 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1],
 *r*4 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1]],
state ↦ [*r*1 ↦ *Switch*, *r*2 ↦ *Leader*, *r*3 ↦ *Follower*, *r*4 ↦ *Follower*],
switchBuffer ↦ [*v*1 ↦ [*term* ↦ 2, *value* ↦ "*v*1", *payload* ↦ "*v*1"],
 *v*2 ↦ [*term* ↦ 2, *value* ↦ "*v*2", *payload* ↦ "*v*2"]],
switchIndex ↦ "*r*1",
switchSentRecord ↦ [*r*1 ↦ {},
 *r*2 ↦ {⟨"*v*1", 2⟩, ⟨"*v*2", 2⟩},
 *r*3 ↦ {⟨"*v*1", 2⟩, ⟨"*v*2", 2⟩},
 *r*4 ↦ {⟨"*v*1", 2⟩}],
unorderedRequests ↦ [*r*1 ↦ {"*v*1", "*v*2"}, *r*2 ↦ {"*v*2"}, *r*3 ↦ {"*v*1", "*v*2"}, *r*4 ↦ {"*v*1"}],
votedFor ↦ [*r*1 ↦ "*r*2", *r*2 ↦ *Nil*, *r*3 ↦ "*r*2", *r*4 ↦ "*r*2"],
voterLog ↦ [*r*1 ↦ ⟨⟩, *r*2 ↦ [*r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩], *r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩],
votesGranted ↦ [*r*1 ↦ {}, *r*2 ↦ {"*r*3", "*r*4"}, *r*3 ↦ {}, *r*4 ↦ {}],
votesResponded ↦ [*r*1 ↦ {}, *r*2 ↦ {"*r*3", "*r*4"}, *r*3 ↦ {}, *r*4 ↦ {}]
],
[
_*TEAction* ↦ [
  position ↦ 10,
  name ↦ "*MySwitchNext*",
  location ↦ "*line* 846, *col* 7 *to line* 847, *col* 60 *of module hovercraft*"
 ],
 Servers ↦ {"*r*2", "*r*3", "*r*4"},
 commitIndex ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
 currentTerm ↦ [*r*1 ↦ 2, *r*2 ↦ 2, *r*3 ↦ 2, *r*4 ↦ 2],
 entryCommitStats ↦ (⟨1, 2⟩ :> [*sentCount* ↦ 0, *ackCount* ↦ 0, *committed* ↦ FALSE] @@
 ⟨2, 2⟩ :> [*sentCount* ↦ 0, *ackCount* ↦ 0, *committed* ↦ FALSE]),
 leaderCount ↦ [*r*1 ↦ 0, *r*2 ↦ 1, *r*3 ↦ 0, *r*4 ↦ 0],
 log ↦ [*r*1 ↦ ⟨⟩,
  *r*2 ↦
     ⟨[*term* ↦ 2, *value* ↦ "*v*1", *payload* ↦ "*v*1"],
       [*term* ↦ 2, *value* ↦ "*v*2", *payload* ↦ "*v*2"]⟩,
  *r*3 ↦ ⟨⟩,
  *r*4 ↦ ⟨⟩],
 matchIndex ↦ [*r*1 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
  *r*2 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],

```
 r3 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
 r4 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0]],
maxc ↦ 2,
messages ↦ ⟨⟩,
nextIndex ↦ [r1 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
 r2 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
 r3 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
 r4 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1]],
state ↦ [r1 ↦ Switch, r2 ↦ Leader, r3 ↦ Follower, r4 ↦ Follower],
switchBuffer ↦ [v1 ↦ [term ↦ 2, value ↦ "v1", payload ↦ "v1"],
 v2 ↦ [term ↦ 2, value ↦ "v2", payload ↦ "v2"]],
switchIndex ↦ "r1",
switchSentRecord ↦ [r1 ↦ {},
 r2 ↦ {⟨"v1", 2⟩, ⟨"v2", 2⟩},
 r3 ↦ {⟨"v1", 2⟩, ⟨"v2", 2⟩},
 r4 ↦ {⟨"v1", 2⟩}],
unorderedRequests ↦ [r1 ↦ {"v1", "v2"}, r2 ↦ {}, r3 ↦ {"v1", "v2"}, r4 ↦ {"v1"}],
votedFor ↦ [r1 ↦ "r2", r2 ↦ Nil, r3 ↦ "r2", r4 ↦ "r2"],
voterLog ↦ [r1 ↦ ⟨⟩, r2 ↦ [r3 ↦ ⟨⟩, r4 ↦ ⟨⟩], r3 ↦ ⟨⟩, r4 ↦ ⟨⟩],
votesGranted ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}],
votesResponded ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}]
],
[
_TEAction ↦ [
  position ↦ 11,
  name ↦ "MySwitchNext",
  location ↦ "line 849, col 7 to line 849, col 56 of module hovercraft"
],
Servers ↦ {"r2", "r3", "r4"},
commitIndex ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
currentTerm ↦ [r1 ↦ 2, r2 ↦ 2, r3 ↦ 2, r4 ↦ 2],
entryCommitStats ↦ (⟨1, 2⟩ :> [sentCount ↦ 1, ackCount ↦ 0, committed ↦ FALSE] @@
 ⟨2, 2⟩ :> [sentCount ↦ 0, ackCount ↦ 0, committed ↦ FALSE]),
leaderCount ↦ [r1 ↦ 0, r2 ↦ 1, r3 ↦ 0, r4 ↦ 0],
log ↦ [r1 ↦ ⟨⟩,
 r2 ↦
    ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"],
      [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
 r3 ↦ ⟨⟩,
 r4 ↦ ⟨⟩],
matchIndex ↦ [r1 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
 r2 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
 r3 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
 r4 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0]],
maxc ↦ 2,
messages ↦ ([mterm ↦ 2,
  mtype ↦ AppendEntriesRequest,
  msource ↦ "r2",
  mdest ↦ "r3",
  mlog ↦
```

```
      ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"],
        [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
   mprevLogIndex ↦ 0,
   mprevLogTerm ↦ 0,
   mentries ↦ ⟨[term ↦ 2, value ↦ "v1"]⟩,
   mcommitIndex ↦ 0] :>
    1),
 nextIndex ↦ [r1 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
  r2 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
  r3 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
  r4 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1]],
 state ↦ [r1 ↦ Switch, r2 ↦ Leader, r3 ↦ Follower, r4 ↦ Follower],
 switchBuffer ↦ [v1 ↦ [term ↦ 2, value ↦ "v1", payload ↦ "v1"],
  v2 ↦ [term ↦ 2, value ↦ "v2", payload ↦ "v2"]],
 switchIndex ↦ "r1",
 switchSentRecord ↦ [r1 ↦ {},
  r2 ↦ {⟨"v1", 2⟩, ⟨"v2", 2⟩},
  r3 ↦ {⟨"v1", 2⟩, ⟨"v2", 2⟩},
  r4 ↦ {⟨"v1", 2⟩}],
 unorderedRequests ↦ [r1 ↦ {"v1", "v2"}, r2 ↦ {}, r3 ↦ {"v1", "v2"}, r4 ↦ {"v1"}],
 votedFor ↦ [r1 ↦ "r2", r2 ↦ Nil, r3 ↦ "r2", r4 ↦ "r2"],
 voterLog ↦ [r1 ↦ ⟨⟩, r2 ↦ [r3 ↦ ⟨⟩, r4 ↦ ⟨⟩], r3 ↦ ⟨⟩, r4 ↦ ⟨⟩],
 votesGranted ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}],
 votesResponded ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}]
],
[
 _TEAction ↦ [
  position ↦ 12,
  name ↦ "MySwitchNext",
  location ↦ "line 850, col 7 to line 851, col 63 of module hovercraft"
 ],
 Servers ↦ {"r2", "r3", "r4"},
 commitIndex ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
 currentTerm ↦ [r1 ↦ 2, r2 ↦ 2, r3 ↦ 2, r4 ↦ 2],
 entryCommitStats ↦ (⟨1, 2⟩ :> [sentCount ↦ 1, ackCount ↦ 0, committed ↦ FALSE] @@
  ⟨2, 2⟩ :> [sentCount ↦ 0, ackCount ↦ 0, committed ↦ FALSE]),
 leaderCount ↦ [r1 ↦ 0, r2 ↦ 1, r3 ↦ 0, r4 ↦ 0],
 log ↦ [r1 ↦ ⟨⟩,
  r2 ↦
     ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"],
       [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
  r3 ↦ ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"]⟩,
  r4 ↦ ⟨⟩],
 matchIndex ↦ [r1 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
  r2 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
  r3 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
  r4 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0]],
 maxc ↦ 2,
 messages ↦ ([mterm ↦ 2,
   mtype ↦ AppendEntriesRequest,
```

```
    msource ↦ "r2",
    mdest ↦ "r3",
    mlog ↦
       ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"],
         [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
    mprevLogIndex ↦ 0,
    mprevLogTerm ↦ 0,
    mentries ↦ ⟨[term ↦ 2, value ↦ "v1"]⟩,
    mcommitIndex ↦ 0] :>
     1),
 nextIndex ↦ [r1 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
  r2 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
  r3 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
  r4 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1]],
 state ↦ [r1 ↦ Switch, r2 ↦ Leader, r3 ↦ Follower, r4 ↦ Follower],
 switchBuffer ↦ [v1 ↦ [term ↦ 2, value ↦ "v1", payload ↦ "v1"],
  v2 ↦ [term ↦ 2, value ↦ "v2", payload ↦ "v2"]],
 switchIndex ↦ "r1",
 switchSentRecord ↦ [r1 ↦ {},
  r2 ↦ {⟨"v1", 2⟩, ⟨"v2", 2⟩},
  r3 ↦ {⟨"v1", 2⟩, ⟨"v2", 2⟩},
  r4 ↦ {⟨"v1", 2⟩}],
 unorderedRequests ↦ [r1 ↦ {"v1", "v2"}, r2 ↦ {}, r3 ↦ {"v2"}, r4 ↦ {"v1"}],
 votedFor ↦ [r1 ↦ "r2", r2 ↦ Nil, r3 ↦ "r2", r4 ↦ "r2"],
 voterLog ↦ [r1 ↦ ⟨⟩, r2 ↦ [r3 ↦ ⟨⟩, r4 ↦ ⟨⟩], r3 ↦ ⟨⟩, r4 ↦ ⟨⟩],
 votesGranted ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}],
 votesResponded ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}]
],
[
 _TEAction ↦ [
  position ↦ 13,
  name ↦ "MySwitchNext",
  location ↦ "line 850, col 7 to line 851, col 63 of module hovercraft"
 ],
 Servers ↦ {"r2", "r3", "r4"},
 commitIndex ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
 currentTerm ↦ [r1 ↦ 2, r2 ↦ 2, r3 ↦ 2, r4 ↦ 2],
 entryCommitStats ↦ (⟨1, 2⟩ :> [sentCount ↦ 1, ackCount ↦ 0, committed ↦ FALSE] @@
  ⟨2, 2⟩ :> [sentCount ↦ 0, ackCount ↦ 0, committed ↦ FALSE]),
 leaderCount ↦ [r1 ↦ 0, r2 ↦ 1, r3 ↦ 0, r4 ↦ 0],
 log ↦ [r1 ↦ ⟨⟩,
  r2 ↦
     ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"],
       [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
  r3 ↦ ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"]⟩,
  r4 ↦ ⟨⟩],
 matchIndex ↦ [r1 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
  r2 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
  r3 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
  r4 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0]],
```

$maxc \mapsto 2,$
$messages \mapsto ([mterm \mapsto 2,$
$mtype \mapsto AppendEntriesResponse,$
$msource \mapsto$ "$r3$",
$mdest \mapsto$ "$r2$",
$msuccess \mapsto$ TRUE,
$mmatchIndex \mapsto 1] :>$
$1 @@$
$[mterm \mapsto 2,$
$mtype \mapsto AppendEntriesRequest,$
$msource \mapsto$ "$r2$",
$mdest \mapsto$ "$r3$",
$mlog \mapsto$
$\langle [term \mapsto 2, value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$[term \mapsto 2, value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]$\rangle,$
$mprevLogIndex \mapsto 0,$
$mprevLogTerm \mapsto 0,$
$mentries \mapsto \langle [term \mapsto 2, value \mapsto$ "$v1$"]$\rangle,$
$mcommitIndex \mapsto 0] :>$
$0),$
$nextIndex \mapsto [r1 \mapsto [r1 \mapsto 1, r2 \mapsto 1, r3 \mapsto 1, r4 \mapsto 1],$
$r2 \mapsto [r1 \mapsto 1, r2 \mapsto 1, r3 \mapsto 1, r4 \mapsto 1],$
$r3 \mapsto [r1 \mapsto 1, r2 \mapsto 1, r3 \mapsto 1, r4 \mapsto 1],$
$r4 \mapsto [r1 \mapsto 1, r2 \mapsto 1, r3 \mapsto 1, r4 \mapsto 1]],$
$state \mapsto [r1 \mapsto Switch, r2 \mapsto Leader, r3 \mapsto Follower, r4 \mapsto Follower],$
$switchBuffer \mapsto [v1 \mapsto [term \mapsto 2, value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$v2 \mapsto [term \mapsto 2, value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]],
$switchIndex \mapsto$ "$r1$",
$switchSentRecord \mapsto [r1 \mapsto \{\},$
$r2 \mapsto \{\langle$"$v1$", $2\rangle, \langle$"$v2$", $2\rangle\},$
$r3 \mapsto \{\langle$"$v1$", $2\rangle, \langle$"$v2$", $2\rangle\},$
$r4 \mapsto \{\langle$"$v1$", $2\rangle\}],$
$unorderedRequests \mapsto [r1 \mapsto \{$"$v1$", "$v2$"$\}, r2 \mapsto \{\}, r3 \mapsto \{$"$v2$"$\}, r4 \mapsto \{$"$v1$"$\}],$
$votedFor \mapsto [r1 \mapsto$ "$r2$", $r2 \mapsto Nil, r3 \mapsto$ "$r2$", $r4 \mapsto$ "$r2$"],
$voterLog \mapsto [r1 \mapsto \langle\rangle, r2 \mapsto [r3 \mapsto \langle\rangle, r4 \mapsto \langle\rangle], r3 \mapsto \langle\rangle, r4 \mapsto \langle\rangle],$
$votesGranted \mapsto [r1 \mapsto \{\}, r2 \mapsto \{$"$r3$", "$r4$"$\}, r3 \mapsto \{\}, r4 \mapsto \{\}],$
$votesResponded \mapsto [r1 \mapsto \{\}, r2 \mapsto \{$"$r3$", "$r4$"$\}, r3 \mapsto \{\}, r4 \mapsto \{\}]$
],
[
$\_TEAction \mapsto [$
$position \mapsto 14,$
$name \mapsto$ "$MySwitchNext$",
$location \mapsto$ "$line$ 850, col 7 $to\ line$ 851, col 63 of $module\ hovercraft$"
],
$Servers \mapsto \{$"$r2$", "$r3$", "$r4$"$\},$
$commitIndex \mapsto [r1 \mapsto 0, r2 \mapsto 0, r3 \mapsto 0, r4 \mapsto 0],$
$currentTerm \mapsto [r1 \mapsto 2, r2 \mapsto 2, r3 \mapsto 2, r4 \mapsto 2],$
$entryCommitStats \mapsto (\langle 1, 2\rangle :> [sentCount \mapsto 1, ackCount \mapsto 1, committed \mapsto$ FALSE$] @@$
$\langle 2, 2\rangle :> [sentCount \mapsto 0, ackCount \mapsto 0, committed \mapsto$ FALSE$]),$
$leaderCount \mapsto [r1 \mapsto 0, r2 \mapsto 1, r3 \mapsto 0, r4 \mapsto 0],$

$log \mapsto [r1 \mapsto \langle\rangle,$
$\ r2 \mapsto$
$\quad \langle[term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$\quad\ [term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]$\rangle$,
$\ r3 \mapsto \langle[term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"]$\rangle$,
$\ r4 \mapsto \langle\rangle]$,
$matchIndex \mapsto [r1 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$\ r2 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 1,\ r4 \mapsto 0]$,
$\ r3 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$\ r4 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]]$,
$maxc \mapsto 2,$
$messages \mapsto ([mterm \mapsto 2,$
$\quad mtype \mapsto AppendEntriesResponse,$
$\quad msource \mapsto$ "$r3$",
$\quad mdest \mapsto$ "$r2$",
$\quad msuccess \mapsto$ TRUE,
$\quad mmatchIndex \mapsto 1] :>$
$\quad\ 0$ @@
$\ [mterm \mapsto 2,$
$\quad mtype \mapsto AppendEntriesRequest,$
$\quad msource \mapsto$ "$r2$",
$\quad mdest \mapsto$ "$r3$",
$\quad mlog \mapsto$
$\quad\ \langle[term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$\quad\quad [term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]$\rangle$,
$\quad mprevLogIndex \mapsto 0,$
$\quad mprevLogTerm \mapsto 0,$
$\quad mentries \mapsto \langle[term \mapsto 2,\ value \mapsto$ "$v1$"]$\rangle$,
$\quad mcommitIndex \mapsto 0] :>$
$\quad\ 0),$
$nextIndex \mapsto [r1 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$\ r2 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 2,\ r4 \mapsto 1]$,
$\ r3 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$\ r4 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]]$,
$state \mapsto [r1 \mapsto Switch,\ r2 \mapsto Leader,\ r3 \mapsto Follower,\ r4 \mapsto Follower]$,
$switchBuffer \mapsto [v1 \mapsto [term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$\ v2 \mapsto [term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]],
$switchIndex \mapsto$ "$r1$",
$switchSentRecord \mapsto [r1 \mapsto \{\}$,
$\ r2 \mapsto \{\langle$"$v1$", $2\rangle, \langle$"$v2$", $2\rangle\}$,
$\ r3 \mapsto \{\langle$"$v1$", $2\rangle, \langle$"$v2$", $2\rangle\}$,
$\ r4 \mapsto \{\langle$"$v1$", $2\rangle\}]$,
$unorderedRequests \mapsto [r1 \mapsto \{$"$v1$", "$v2$"$\}, r2 \mapsto \{\}, r3 \mapsto \{$"$v2$"$\}, r4 \mapsto \{$"$v1$"$\}]$,
$votedFor \mapsto [r1 \mapsto$ "$r2$", $r2 \mapsto Nil,\ r3 \mapsto$ "$r2$", $r4 \mapsto$ "$r2$"],
$voterLog \mapsto [r1 \mapsto \langle\rangle,\ r2 \mapsto [r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle],\ r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle]$,
$votesGranted \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{$"$r3$", "$r4$"$\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$,
$votesResponded \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{$"$r3$", "$r4$"$\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$
],
[
$\_TEAction \mapsto [$

$position \mapsto 15,$
$name \mapsto$ "$MySwitchNext$",
$location \mapsto$ "$line$ 848, col 7 $to\ line$ 848, col 46 $of\ module\ hovercraft$"
],
$Servers \mapsto \{$"$r2$", "$r3$", "$r4$"$\}$,
$commitIndex \mapsto [r1 \mapsto 0,\ r2 \mapsto 1,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$currentTerm \mapsto [r1 \mapsto 2,\ r2 \mapsto 2,\ r3 \mapsto 2,\ r4 \mapsto 2]$,
$entryCommitStats \mapsto (\langle 1, 2\rangle :> [sentCount \mapsto 1,\ ackCount \mapsto 1,\ committed \mapsto$ TRUE$]$ @@
$\langle 2, 2\rangle :> [sentCount \mapsto 0,\ ackCount \mapsto 0,\ committed \mapsto$ FALSE$])$,
$leaderCount \mapsto [r1 \mapsto 0,\ r2 \mapsto 1,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$log \mapsto [r1 \mapsto \langle\rangle,$
$r2 \mapsto$
$\langle[term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"$]$,
$[term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"$]\rangle$,
$r3 \mapsto \langle[term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"$]\rangle$,
$r4 \mapsto \langle\rangle]$,
$matchIndex \mapsto [r1 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$r2 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 1,\ r4 \mapsto 0]$,
$r3 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$r4 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]]$,
$maxc \mapsto 2,$
$messages \mapsto ([mterm \mapsto 2,$
$mtype \mapsto AppendEntriesResponse,$
$msource \mapsto$ "$r3$",
$mdest \mapsto$ "$r2$",
$msuccess \mapsto$ TRUE,
$mmatchIndex \mapsto 1] :>$
0 @@
$[mterm \mapsto 2,$
$mtype \mapsto AppendEntriesRequest,$
$msource \mapsto$ "$r2$",
$mdest \mapsto$ "$r3$",
$mlog \mapsto$
$\langle[term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"$]$,
$[term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"$]\rangle$,
$mprevLogIndex \mapsto 0,$
$mprevLogTerm \mapsto 0,$
$mentries \mapsto \langle[term \mapsto 2,\ value \mapsto$ "$v1$"$]\rangle$,
$mcommitIndex \mapsto 0] :>$
$0),$
$nextIndex \mapsto [r1 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r2 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 2,\ r4 \mapsto 1]$,
$r3 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]$,
$r4 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]]$,
$state \mapsto [r1 \mapsto Switch,\ r2 \mapsto Leader,\ r3 \mapsto Follower,\ r4 \mapsto Follower]$,
$switchBuffer \mapsto [v1 \mapsto [term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"$]$,
$v2 \mapsto [term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"$]]$,
$switchIndex \mapsto$ "$r1$",
$switchSentRecord \mapsto [r1 \mapsto \{\}$,
$r2 \mapsto \{\langle$"$v1$", $2\rangle$, $\langle$"$v2$", $2\rangle\}$,

```
 r3 ↦ {⟨“v1”, 2⟩, ⟨“v2”, 2⟩},
 r4 ↦ {⟨“v1”, 2⟩}],
unorderedRequests ↦ [r1 ↦ {“v1”, “v2”}, r2 ↦ {}, r3 ↦ {“v2”}, r4 ↦ {“v1”}],
votedFor ↦ [r1 ↦ “r2”, r2 ↦ Nil, r3 ↦ “r2”, r4 ↦ “r2”],
voterLog ↦ [r1 ↦ ⟨⟩, r2 ↦ [r3 ↦ ⟨⟩, r4 ↦ ⟨⟩], r3 ↦ ⟨⟩, r4 ↦ ⟨⟩],
votesGranted ↦ [r1 ↦ {}, r2 ↦ {“r3”, “r4”}, r3 ↦ {}, r4 ↦ {}],
votesResponded ↦ [r1 ↦ {}, r2 ↦ {“r3”, “r4”}, r3 ↦ {}, r4 ↦ {}]
],
[
_TEAction ↦ [
  position ↦ 16,
  name ↦ “MySwitchNext”,
  location ↦ “line 849, col 7 to line 849, col 56 of module hovercraft”
 ],
Servers ↦ {“r2”, “r3”, “r4”},
commitIndex ↦ [r1 ↦ 0, r2 ↦ 1, r3 ↦ 0, r4 ↦ 0],
currentTerm ↦ [r1 ↦ 2, r2 ↦ 2, r3 ↦ 2, r4 ↦ 2],
entryCommitStats ↦ (⟨1, 2⟩ :> [sentCount ↦ 1, ackCount ↦ 1, committed ↦ TRUE] @@
 ⟨2, 2⟩ :> [sentCount ↦ 1, ackCount ↦ 0, committed ↦ FALSE]),
leaderCount ↦ [r1 ↦ 0, r2 ↦ 1, r3 ↦ 0, r4 ↦ 0],
log ↦ [r1 ↦ ⟨⟩,
 r2 ↦
    ⟨[term ↦ 2, value ↦ “v1”, payload ↦ “v1”],
      [term ↦ 2, value ↦ “v2”, payload ↦ “v2”]⟩,
 r3 ↦ ⟨[term ↦ 2, value ↦ “v1”, payload ↦ “v1”]⟩,
 r4 ↦ ⟨⟩],
matchIndex ↦ [r1 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
 r2 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 1, r4 ↦ 0],
 r3 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
 r4 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0]],
maxc ↦ 2,
messages ↦ ([mterm ↦ 2,
   mtype ↦ AppendEntriesResponse,
   msource ↦ “r3”,
   mdest ↦ “r2”,
   msuccess ↦ TRUE,
   mmatchIndex ↦ 1] :>
    0 @@
 [mterm ↦ 2,
   mtype ↦ AppendEntriesRequest,
   msource ↦ “r2”,
   mdest ↦ “r3”,
   mlog ↦
      ⟨[term ↦ 2, value ↦ “v1”, payload ↦ “v1”],
        [term ↦ 2, value ↦ “v2”, payload ↦ “v2”]⟩,
   mprevLogIndex ↦ 0,
   mprevLogTerm ↦ 0,
   mentries ↦ ⟨[term ↦ 2, value ↦ “v1”]⟩,
   mcommitIndex ↦ 0] :>
    0 @@
```

[*mterm* ↦ 2,
mtype ↦ *AppendEntriesRequest*,
msource ↦ “*r*2”,
mdest ↦ “*r*3”,
mlog ↦
⟨[*term* ↦ 2, *value* ↦ “*v*1”, *payload* ↦ “*v*1”],
[*term* ↦ 2, *value* ↦ “*v*2”, *payload* ↦ “*v*2”]⟩,
mprevLogIndex ↦ 1,
mprevLogTerm ↦ 2,
mentries ↦ ⟨[*term* ↦ 2, *value* ↦ “*v*2”]⟩,
mcommitIndex ↦ 1] :>
1),
nextIndex ↦ [*r*1 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1],
*r*2 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 2, *r*4 ↦ 1],
*r*3 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1],
*r*4 ↦ [*r*1 ↦ 1, *r*2 ↦ 1, *r*3 ↦ 1, *r*4 ↦ 1]],
state ↦ [*r*1 ↦ *Switch*, *r*2 ↦ *Leader*, *r*3 ↦ *Follower*, *r*4 ↦ *Follower*],
switchBuffer ↦ [*v*1 ↦ [*term* ↦ 2, *value* ↦ “*v*1”, *payload* ↦ “*v*1”],
*v*2 ↦ [*term* ↦ 2, *value* ↦ “*v*2”, *payload* ↦ “*v*2”]],
switchIndex ↦ “*r*1”,
switchSentRecord ↦ [*r*1 ↦ {},
*r*2 ↦ {⟨“*v*1”, 2⟩, ⟨“*v*2”, 2⟩},
*r*3 ↦ {⟨“*v*1”, 2⟩, ⟨“*v*2”, 2⟩},
*r*4 ↦ {⟨“*v*1”, 2⟩}],
unorderedRequests ↦ [*r*1 ↦ {“*v*1”, “*v*2”}, *r*2 ↦ {}, *r*3 ↦ {“*v*2”}, *r*4 ↦ {“*v*1”}],
votedFor ↦ [*r*1 ↦ “*r*2”, *r*2 ↦ *Nil*, *r*3 ↦ “*r*2”, *r*4 ↦ “*r*2”],
voterLog ↦ [*r*1 ↦ ⟨⟩, *r*2 ↦ [*r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩], *r*3 ↦ ⟨⟩, *r*4 ↦ ⟨⟩],
votesGranted ↦ [*r*1 ↦ {}, *r*2 ↦ {“*r*3”, “*r*4”}, *r*3 ↦ {}, *r*4 ↦ {}],
votesResponded ↦ [*r*1 ↦ {}, *r*2 ↦ {“*r*3”, “*r*4”}, *r*3 ↦ {}, *r*4 ↦ {}]
],
[
_*TEAction* ↦ [
position ↦ 17,
name ↦ “*MySwitchNext*”,
location ↦ “*line* 849, *col* 7 *to line* 849, *col* 56 *of module hovercraft*”
],
Servers ↦ {“*r*2”, “*r*3”, “*r*4”},
commitIndex ↦ [*r*1 ↦ 0, *r*2 ↦ 1, *r*3 ↦ 0, *r*4 ↦ 0],
currentTerm ↦ [*r*1 ↦ 2, *r*2 ↦ 2, *r*3 ↦ 2, *r*4 ↦ 2],
entryCommitStats ↦ (⟨1, 2⟩ :> [*sentCount* ↦ 1, *ackCount* ↦ 1, *committed* ↦ TRUE] @@
⟨2, 2⟩ :> [*sentCount* ↦ 1, *ackCount* ↦ 0, *committed* ↦ FALSE]),
leaderCount ↦ [*r*1 ↦ 0, *r*2 ↦ 1, *r*3 ↦ 0, *r*4 ↦ 0],
log ↦ [*r*1 ↦ ⟨⟩,
*r*2 ↦
⟨[*term* ↦ 2, *value* ↦ “*v*1”, *payload* ↦ “*v*1”],
[*term* ↦ 2, *value* ↦ “*v*2”, *payload* ↦ “*v*2”]⟩,
*r*3 ↦ ⟨[*term* ↦ 2, *value* ↦ “*v*1”, *payload* ↦ “*v*1”]⟩,
*r*4 ↦ ⟨⟩],
matchIndex ↦ [*r*1 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 0, *r*4 ↦ 0],
*r*2 ↦ [*r*1 ↦ 0, *r*2 ↦ 0, *r*3 ↦ 1, *r*4 ↦ 0],

$r3 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0],$
$r4 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]],$
$maxc \mapsto 2,$
$messages \mapsto ([mterm \mapsto 2,$
$mtype \mapsto AppendEntriesResponse,$
$msource \mapsto$ "$r3$",
$mdest \mapsto$ "$r2$",
$msuccess \mapsto$ TRUE,
$mmatchIndex \mapsto 1] :>$
$0 @@$
$[mterm \mapsto 2,$
$mtype \mapsto AppendEntriesRequest,$
$msource \mapsto$ "$r2$",
$mdest \mapsto$ "$r3$",
$mlog \mapsto$
$\langle [term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$[term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]$\rangle,$
$mprevLogIndex \mapsto 0,$
$mprevLogTerm \mapsto 0,$
$mentries \mapsto \langle [term \mapsto 2,\ value \mapsto$ "$v1$"]$\rangle,$
$mcommitIndex \mapsto 0] :>$
$0 @@$
$[mterm \mapsto 2,$
$mtype \mapsto AppendEntriesRequest,$
$msource \mapsto$ "$r2$",
$mdest \mapsto$ "$r3$",
$mlog \mapsto$
$\langle [term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$[term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]$\rangle,$
$mprevLogIndex \mapsto 1,$
$mprevLogTerm \mapsto 2,$
$mentries \mapsto \langle [term \mapsto 2,\ value \mapsto$ "$v2$"]$\rangle,$
$mcommitIndex \mapsto 1] :>$
$1 @@$
$[mterm \mapsto 2,$
$mtype \mapsto AppendEntriesRequest,$
$msource \mapsto$ "$r2$",
$mdest \mapsto$ "$r4$",
$mlog \mapsto$
$\langle [term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$[term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]$\rangle,$
$mprevLogIndex \mapsto 0,$
$mprevLogTerm \mapsto 0,$
$mentries \mapsto \langle [term \mapsto 2,\ value \mapsto$ "$v1$"]$\rangle,$
$mcommitIndex \mapsto 1] :>$
$1),$
$nextIndex \mapsto [r1 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1],$
$r2 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 2,\ r4 \mapsto 1],$
$r3 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1],$
$r4 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]],$

$state \mapsto [r1 \mapsto Switch,\ r2 \mapsto Leader,\ r3 \mapsto Follower,\ r4 \mapsto Follower]$,
$switchBuffer \mapsto [v1 \mapsto [term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}]$,
$\ v2 \mapsto [term \mapsto 2,\ value \mapsto \text{“}v2\text{”},\ payload \mapsto \text{“}v2\text{”}]]$,
$switchIndex \mapsto \text{“}r1\text{”}$,
$switchSentRecord \mapsto [r1 \mapsto \{\}$,
$\ r2 \mapsto \{\langle \text{“}v1\text{”},\ 2\rangle,\ \langle \text{“}v2\text{”},\ 2\rangle\}$,
$\ r3 \mapsto \{\langle \text{“}v1\text{”},\ 2\rangle,\ \langle \text{“}v2\text{”},\ 2\rangle\}$,
$\ r4 \mapsto \{\langle \text{“}v1\text{”},\ 2\rangle\}]$,
$unorderedRequests \mapsto [r1 \mapsto \{\text{“}v1\text{”},\ \text{“}v2\text{”}\},\ r2 \mapsto \{\},\ r3 \mapsto \{\text{“}v2\text{”}\},\ r4 \mapsto \{\text{“}v1\text{”}\}]$,
$votedFor \mapsto [r1 \mapsto \text{“}r2\text{”},\ r2 \mapsto Nil,\ r3 \mapsto \text{“}r2\text{”},\ r4 \mapsto \text{“}r2\text{”}]$,
$voterLog \mapsto [r1 \mapsto \langle\rangle,\ r2 \mapsto [r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle],\ r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle]$,
$votesGranted \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{\text{“}r3\text{”},\ \text{“}r4\text{”}\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$,
$votesResponded \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{\text{“}r3\text{”},\ \text{“}r4\text{”}\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$
],
[
$\_TEAction \mapsto [$
$\ position \mapsto 18,$
$\ name \mapsto \text{“}MySwitchNext\text{”},$
$\ location \mapsto \text{“}line\ 850,\ col\ 7\ to\ line\ 851,\ col\ 63\ of\ module\ hovercraft\text{”}$
],
$Servers \mapsto \{\text{“}r2\text{”},\ \text{“}r3\text{”},\ \text{“}r4\text{”}\}$,
$commitIndex \mapsto [r1 \mapsto 0,\ r2 \mapsto 1,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$currentTerm \mapsto [r1 \mapsto 2,\ r2 \mapsto 2,\ r3 \mapsto 2,\ r4 \mapsto 2]$,
$entryCommitStats \mapsto (\langle 1,\ 2\rangle :> [sentCount \mapsto 1,\ ackCount \mapsto 1,\ committed \mapsto \text{TRUE}]\ @@$
$\ \langle 2,\ 2\rangle :> [sentCount \mapsto 1,\ ackCount \mapsto 0,\ committed \mapsto \text{FALSE}])$,
$leaderCount \mapsto [r1 \mapsto 0,\ r2 \mapsto 1,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$log \mapsto [r1 \mapsto \langle\rangle,$
$\ r2 \mapsto$
$\quad \langle [term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}],$
$\quad\ [term \mapsto 2,\ value \mapsto \text{“}v2\text{”},\ payload \mapsto \text{“}v2\text{”}]\rangle,$
$\ r3 \mapsto$
$\quad \langle [term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}],$
$\quad\ [term \mapsto 2,\ value \mapsto \text{“}v2\text{”},\ payload \mapsto \text{“}v2\text{”}]\rangle,$
$\ r4 \mapsto \langle\rangle]$,
$matchIndex \mapsto [r1 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$\ r2 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 1,\ r4 \mapsto 0]$,
$\ r3 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$\ r4 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]]$,
$maxc \mapsto 2$,
$messages \mapsto ([mterm \mapsto 2,$
$\quad mtype \mapsto AppendEntriesResponse,$
$\quad msource \mapsto \text{“}r3\text{”},$
$\quad mdest \mapsto \text{“}r2\text{”},$
$\quad msuccess \mapsto \text{TRUE},$
$\quad mmatchIndex \mapsto 1] :>$
$\quad\ 0\ @@$
$\ [mterm \mapsto 2,$
$\quad mtype \mapsto AppendEntriesRequest,$
$\quad msource \mapsto \text{“}r2\text{”},$
$\quad mdest \mapsto \text{“}r3\text{”},$

$mlog \mapsto$
$\langle[term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}],$
$[term \mapsto 2,\ value \mapsto \text{“}v2\text{”},\ payload \mapsto \text{“}v2\text{”}]\rangle,$
$mprevLogIndex \mapsto 0,$
$mprevLogTerm \mapsto 0,$
$mentries \mapsto \langle[term \mapsto 2,\ value \mapsto \text{“}v1\text{”}]\rangle,$
$mcommitIndex \mapsto 0]:>$
$0\ @@$
$[mterm \mapsto 2,$
$mtype \mapsto AppendEntriesRequest,$
$msource \mapsto \text{“}r2\text{”},$
$mdest \mapsto \text{“}r3\text{”},$
$mlog \mapsto$
$\langle[term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}],$
$[term \mapsto 2,\ value \mapsto \text{“}v2\text{”},\ payload \mapsto \text{“}v2\text{”}]\rangle,$
$mprevLogIndex \mapsto 1,$
$mprevLogTerm \mapsto 2,$
$mentries \mapsto \langle[term \mapsto 2,\ value \mapsto \text{“}v2\text{”}]\rangle,$
$mcommitIndex \mapsto 1]:>$
$1\ @@$
$[mterm \mapsto 2,$
$mtype \mapsto AppendEntriesRequest,$
$msource \mapsto \text{“}r2\text{”},$
$mdest \mapsto \text{“}r4\text{”},$
$mlog \mapsto$
$\langle[term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}],$
$[term \mapsto 2,\ value \mapsto \text{“}v2\text{”},\ payload \mapsto \text{“}v2\text{”}]\rangle,$
$mprevLogIndex \mapsto 0,$
$mprevLogTerm \mapsto 0,$
$mentries \mapsto \langle[term \mapsto 2,\ value \mapsto \text{“}v1\text{”}]\rangle,$
$mcommitIndex \mapsto 1]:>$
$1),$
$nextIndex \mapsto [r1 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1],$
$r2 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 2,\ r4 \mapsto 1],$
$r3 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1],$
$r4 \mapsto [r1 \mapsto 1,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 1]],$
$state \mapsto [r1 \mapsto Switch,\ r2 \mapsto Leader,\ r3 \mapsto Follower,\ r4 \mapsto Follower],$
$switchBuffer \mapsto [v1 \mapsto [term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}],$
$v2 \mapsto [term \mapsto 2,\ value \mapsto \text{“}v2\text{”},\ payload \mapsto \text{“}v2\text{”}]],$
$switchIndex \mapsto \text{“}r1\text{”},$
$switchSentRecord \mapsto [r1 \mapsto \{\},$
$r2 \mapsto \{\langle\text{“}v1\text{”}, 2\rangle, \langle\text{“}v2\text{”}, 2\rangle\},$
$r3 \mapsto \{\langle\text{“}v1\text{”}, 2\rangle, \langle\text{“}v2\text{”}, 2\rangle\},$
$r4 \mapsto \{\langle\text{“}v1\text{”}, 2\rangle\}],$
$unorderedRequests \mapsto [r1 \mapsto \{\text{“}v1\text{”},\ \text{“}v2\text{”}\},\ r2 \mapsto \{\},\ r3 \mapsto \{\},\ r4 \mapsto \{\text{“}v1\text{”}\}],$
$votedFor \mapsto [r1 \mapsto \text{“}r2\text{”},\ r2 \mapsto Nil,\ r3 \mapsto \text{“}r2\text{”},\ r4 \mapsto \text{“}r2\text{”}],$
$voterLog \mapsto [r1 \mapsto \langle\rangle,\ r2 \mapsto [r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle],\ r3 \mapsto \langle\rangle,\ r4 \mapsto \langle\rangle],$
$votesGranted \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{\text{“}r3\text{”},\ \text{“}r4\text{”}\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}],$
$votesResponded \mapsto [r1 \mapsto \{\},\ r2 \mapsto \{\text{“}r3\text{”},\ \text{“}r4\text{”}\},\ r3 \mapsto \{\},\ r4 \mapsto \{\}]$
],

```
[
 _TEAction ↦ [
   position ↦ 19,
   name ↦ "MySwitchNext",
   location ↦ "line 850, col 7 to line 851, col 63 of module hovercraft"
 ],
 Servers ↦ {"r2", "r3", "r4"},
 commitIndex ↦ [r1 ↦ 0, r2 ↦ 1, r3 ↦ 1, r4 ↦ 0],
 currentTerm ↦ [r1 ↦ 2, r2 ↦ 2, r3 ↦ 2, r4 ↦ 2],
 entryCommitStats ↦ (⟨1, 2⟩ :> [sentCount ↦ 1, ackCount ↦ 1, committed ↦ TRUE] @@
  ⟨2, 2⟩ :> [sentCount ↦ 1, ackCount ↦ 0, committed ↦ FALSE]),
 leaderCount ↦ [r1 ↦ 0, r2 ↦ 1, r3 ↦ 0, r4 ↦ 0],
 log ↦ [r1 ↦ ⟨⟩,
  r2 ↦
     ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"],
       [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
  r3 ↦
     ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"],
       [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
  r4 ↦ ⟨⟩],
 matchIndex ↦ [r1 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
  r2 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 1, r4 ↦ 0],
  r3 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0],
  r4 ↦ [r1 ↦ 0, r2 ↦ 0, r3 ↦ 0, r4 ↦ 0]],
 maxc ↦ 2,
 messages ↦ ([mterm ↦ 2,
    mtype ↦ AppendEntriesResponse,
    msource ↦ "r3",
    mdest ↦ "r2",
    msuccess ↦ TRUE,
    mmatchIndex ↦ 1] :>
     0 @@
  [mterm ↦ 2,
    mtype ↦ AppendEntriesResponse,
    msource ↦ "r3",
    mdest ↦ "r2",
    msuccess ↦ TRUE,
    mmatchIndex ↦ 2] :>
     1 @@
  [mterm ↦ 2,
    mtype ↦ AppendEntriesRequest,
    msource ↦ "r2",
    mdest ↦ "r3",
    mlog ↦
       ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"],
         [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
    mprevLogIndex ↦ 0,
    mprevLogTerm ↦ 0,
    mentries ↦ ⟨[term ↦ 2, value ↦ "v1"]⟩,
    mcommitIndex ↦ 0] :>
```

```
    0 @@
  [mterm ↦ 2,
    mtype ↦ AppendEntriesRequest,
    msource ↦ "r2",
    mdest ↦ "r3",
    mlog ↦
      ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"],
        [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
    mprevLogIndex ↦ 1,
    mprevLogTerm ↦ 2,
    mentries ↦ ⟨[term ↦ 2, value ↦ "v2"]⟩,
    mcommitIndex ↦ 1]:>
    0 @@
  [mterm ↦ 2,
    mtype ↦ AppendEntriesRequest,
    msource ↦ "r2",
    mdest ↦ "r4",
    mlog ↦
      ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"],
        [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
    mprevLogIndex ↦ 0,
    mprevLogTerm ↦ 0,
    mentries ↦ ⟨[term ↦ 2, value ↦ "v1"]⟩,
    mcommitIndex ↦ 1]:>
    1),
 nextIndex ↦ [r1 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
  r2 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 2, r4 ↦ 1],
  r3 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
  r4 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1]],
 state ↦ [r1 ↦ Switch, r2 ↦ Leader, r3 ↦ Follower, r4 ↦ Follower],
 switchBuffer ↦ [v1 ↦ [term ↦ 2, value ↦ "v1", payload ↦ "v1"],
  v2 ↦ [term ↦ 2, value ↦ "v2", payload ↦ "v2"]],
 switchIndex ↦ "r1",
 switchSentRecord ↦ [r1 ↦ {},
  r2 ↦ {⟨"v1", 2⟩, ⟨"v2", 2⟩},
  r3 ↦ {⟨"v1", 2⟩, ⟨"v2", 2⟩},
  r4 ↦ {⟨"v1", 2⟩}],
 unorderedRequests ↦ [r1 ↦ {"v1", "v2"}, r2 ↦ {}, r3 ↦ {}, r4 ↦ {"v1"}],
 votedFor ↦ [r1 ↦ "r2", r2 ↦ Nil, r3 ↦ "r2", r4 ↦ "r2"],
 voterLog ↦ [r1 ↦ ⟨⟩, r2 ↦ [r3 ↦ ⟨⟩, r4 ↦ ⟨⟩], r3 ↦ ⟨⟩, r4 ↦ ⟨⟩],
 votesGranted ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}],
 votesResponded ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}]
],
[
 _TEAction ↦ [
  position ↦ 20,
  name ↦ "MySwitchNext",
  location ↦ "line 850, col 7 to line 851, col 63 of module hovercraft"
 ],
 Servers ↦ {"r2", "r3", "r4"},
```

$commitIndex \mapsto [r1 \mapsto 0,\ r2 \mapsto 1,\ r3 \mapsto 1,\ r4 \mapsto 0]$,
$currentTerm \mapsto [r1 \mapsto 2,\ r2 \mapsto 2,\ r3 \mapsto 2,\ r4 \mapsto 2]$,
$entryCommitStats \mapsto (\langle 1,\ 2\rangle :> [sentCount \mapsto 1,\ ackCount \mapsto 1,\ committed \mapsto \text{TRUE}]\ @@$
$\langle 2,\ 2\rangle :> [sentCount \mapsto 1,\ ackCount \mapsto 0,\ committed \mapsto \text{FALSE}])$,
$leaderCount \mapsto [r1 \mapsto 0,\ r2 \mapsto 1,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$log \mapsto [r1 \mapsto \langle\rangle$,
$r2 \mapsto$
$\langle [term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$[term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]$\rangle$,
$r3 \mapsto$
$\langle [term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$[term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]$\rangle$,
$r4 \mapsto \langle [term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"]$\rangle]$,
$matchIndex \mapsto [r1 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$r2 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 1,\ r4 \mapsto 0]$,
$r3 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]$,
$r4 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]]$,
$maxc \mapsto 2$,
$messages \mapsto ([mterm \mapsto 2$,
$mtype \mapsto AppendEntriesResponse$,
$msource \mapsto$ "$r3$",
$mdest \mapsto$ "$r2$",
$msuccess \mapsto \text{TRUE}$,
$mmatchIndex \mapsto 1] :>$
$0\ @@$
$[mterm \mapsto 2$,
$mtype \mapsto AppendEntriesResponse$,
$msource \mapsto$ "$r3$",
$mdest \mapsto$ "$r2$",
$msuccess \mapsto \text{TRUE}$,
$mmatchIndex \mapsto 2] :>$
$1\ @@$
$[mterm \mapsto 2$,
$mtype \mapsto AppendEntriesRequest$,
$msource \mapsto$ "$r2$",
$mdest \mapsto$ "$r3$",
$mlog \mapsto$
$\langle [term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],
$[term \mapsto 2,\ value \mapsto$ "$v2$", $payload \mapsto$ "$v2$"]$\rangle$,
$mprevLogIndex \mapsto 0$,
$mprevLogTerm \mapsto 0$,
$mentries \mapsto \langle [term \mapsto 2,\ value \mapsto$ "$v1$"]$\rangle$,
$mcommitIndex \mapsto 0] :>$
$0\ @@$
$[mterm \mapsto 2$,
$mtype \mapsto AppendEntriesRequest$,
$msource \mapsto$ "$r2$",
$mdest \mapsto$ "$r3$",
$mlog \mapsto$
$\langle [term \mapsto 2,\ value \mapsto$ "$v1$", $payload \mapsto$ "$v1$"],

```
        [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
    mprevLogIndex ↦ 1,
    mprevLogTerm ↦ 2,
    mentries ↦ ⟨[term ↦ 2, value ↦ "v2"]⟩,
    mcommitIndex ↦ 1]:>
     0 @@
  [mterm ↦ 2,
    mtype ↦ AppendEntriesRequest,
    msource ↦ "r2",
    mdest ↦ "r4",
    mlog ↦
       ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"],
         [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
    mprevLogIndex ↦ 0,
    mprevLogTerm ↦ 0,
    mentries ↦ ⟨[term ↦ 2, value ↦ "v1"]⟩,
    mcommitIndex ↦ 1]:>
     1),
 nextIndex ↦ [r1 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
  r2 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 2, r4 ↦ 1],
  r3 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
  r4 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1]],
 state ↦ [r1 ↦ Switch, r2 ↦ Leader, r3 ↦ Follower, r4 ↦ Follower],
 switchBuffer ↦ [v1 ↦ [term ↦ 2, value ↦ "v1", payload ↦ "v1"],
  v2 ↦ [term ↦ 2, value ↦ "v2", payload ↦ "v2"]],
 switchIndex ↦ "r1",
 switchSentRecord ↦ [r1 ↦ {},
  r2 ↦ {⟨"v1", 2⟩, ⟨"v2", 2⟩},
  r3 ↦ {⟨"v1", 2⟩, ⟨"v2", 2⟩},
  r4 ↦ {⟨"v1", 2⟩}],
 unorderedRequests ↦ [r1 ↦ {"v1", "v2"}, r2 ↦ {}, r3 ↦ {}, r4 ↦ {}],
 votedFor ↦ [r1 ↦ "r2", r2 ↦ Nil, r3 ↦ "r2", r4 ↦ "r2"],
 voterLog ↦ [r1 ↦ ⟨⟩, r2 ↦ [r3 ↦ ⟨⟩, r4 ↦ ⟨⟩], r3 ↦ ⟨⟩, r4 ↦ ⟨⟩],
 votesGranted ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}],
 votesResponded ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}]
],
[
 _TEAction ↦ [
   position ↦ 21,
   name ↦ "MySwitchNext",
   location ↦ "line 850, col 7 to line 851, col 63 of module hovercraft"
 ],
 Servers ↦ {"r2", "r3", "r4"},
 commitIndex ↦ [r1 ↦ 0, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
 currentTerm ↦ [r1 ↦ 2, r2 ↦ 2, r3 ↦ 2, r4 ↦ 2],
 entryCommitStats ↦ (⟨1, 2⟩:> [sentCount ↦ 1, ackCount ↦ 1, committed ↦ TRUE] @@
  ⟨2, 2⟩:> [sentCount ↦ 1, ackCount ↦ 0, committed ↦ FALSE]),
 leaderCount ↦ [r1 ↦ 0, r2 ↦ 1, r3 ↦ 0, r4 ↦ 0],
 log ↦ [r1 ↦ ⟨⟩,
  r2 ↦
```

$\langle[term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}],$
$[term \mapsto 2,\ value \mapsto \text{“}v2\text{”},\ payload \mapsto \text{“}v2\text{”}]\rangle,$
$r3 \mapsto$
$\langle[term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}],$
$[term \mapsto 2,\ value \mapsto \text{“}v2\text{”},\ payload \mapsto \text{“}v2\text{”}]\rangle,$
$r4 \mapsto \langle[term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}]\rangle],$
$matchIndex \mapsto [r1 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0],$
$r2 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 1,\ r4 \mapsto 0],$
$r3 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0],$
$r4 \mapsto [r1 \mapsto 0,\ r2 \mapsto 0,\ r3 \mapsto 0,\ r4 \mapsto 0]],$
$maxc \mapsto 2,$
$messages \mapsto ([mterm \mapsto 2,$
$mtype \mapsto AppendEntriesResponse,$
$msource \mapsto \text{“}r3\text{”},$
$mdest \mapsto \text{“}r2\text{”},$
$msuccess \mapsto \text{TRUE},$
$mmatchIndex \mapsto 1] :>$
$0 @@$
$[mterm \mapsto 2,$
$mtype \mapsto AppendEntriesResponse,$
$msource \mapsto \text{“}r3\text{”},$
$mdest \mapsto \text{“}r2\text{”},$
$msuccess \mapsto \text{TRUE},$
$mmatchIndex \mapsto 2] :>$
$1 @@$
$[mterm \mapsto 2,$
$mtype \mapsto AppendEntriesResponse,$
$msource \mapsto \text{“}r4\text{”},$
$mdest \mapsto \text{“}r2\text{”},$
$msuccess \mapsto \text{TRUE},$
$mmatchIndex \mapsto 1] :>$
$1 @@$
$[mterm \mapsto 2,$
$mtype \mapsto AppendEntriesRequest,$
$msource \mapsto \text{“}r2\text{”},$
$mdest \mapsto \text{“}r3\text{”},$
$mlog \mapsto$
$\langle[term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}],$
$[term \mapsto 2,\ value \mapsto \text{“}v2\text{”},\ payload \mapsto \text{“}v2\text{”}]\rangle,$
$mprevLogIndex \mapsto 0,$
$mprevLogTerm \mapsto 0,$
$mentries \mapsto \langle[term \mapsto 2,\ value \mapsto \text{“}v1\text{”}]\rangle,$
$mcommitIndex \mapsto 0] :>$
$0 @@$
$[mterm \mapsto 2,$
$mtype \mapsto AppendEntriesRequest,$
$msource \mapsto \text{“}r2\text{”},$
$mdest \mapsto \text{“}r3\text{”},$
$mlog \mapsto$
$\langle[term \mapsto 2,\ value \mapsto \text{“}v1\text{”},\ payload \mapsto \text{“}v1\text{”}],$

```
        [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
    mprevLogIndex ↦ 1,
    mprevLogTerm ↦ 2,
    mentries ↦ ⟨[term ↦ 2, value ↦ "v2"]⟩,
    mcommitIndex ↦ 1]:>
     0 @@
  [mterm ↦ 2,
    mtype ↦ AppendEntriesRequest,
    msource ↦ "r2",
    mdest ↦ "r4",
    mlog ↦
       ⟨[term ↦ 2, value ↦ "v1", payload ↦ "v1"],
         [term ↦ 2, value ↦ "v2", payload ↦ "v2"]⟩,
    mprevLogIndex ↦ 0,
    mprevLogTerm ↦ 0,
    mentries ↦ ⟨[term ↦ 2, value ↦ "v1"]⟩,
    mcommitIndex ↦ 1]:>
     0),
  nextIndex ↦ [r1 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
   r2 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 2, r4 ↦ 1],
   r3 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1],
   r4 ↦ [r1 ↦ 1, r2 ↦ 1, r3 ↦ 1, r4 ↦ 1]],
  state ↦ [r1 ↦ Switch, r2 ↦ Leader, r3 ↦ Follower, r4 ↦ Follower],
  switchBuffer ↦ [v1 ↦ [term ↦ 2, value ↦ "v1", payload ↦ "v1"],
   v2 ↦ [term ↦ 2, value ↦ "v2", payload ↦ "v2"]],
  switchIndex ↦ "r1",
  switchSentRecord ↦ [r1 ↦ {},
   r2 ↦ {⟨"v1", 2⟩, ⟨"v2", 2⟩},
   r3 ↦ {⟨"v1", 2⟩, ⟨"v2", 2⟩},
   r4 ↦ {⟨"v1", 2⟩}],
  unorderedRequests ↦ [r1 ↦ {"v1", "v2"}, r2 ↦ {}, r3 ↦ {}, r4 ↦ {}],
  votedFor ↦ [r1 ↦ "r2", r2 ↦ Nil, r3 ↦ "r2", r4 ↦ "r2"],
  voterLog ↦ [r1 ↦ ⟨⟩, r2 ↦ [r3 ↦ ⟨⟩, r4 ↦ ⟨⟩], r3 ↦ ⟨⟩, r4 ↦ ⟨⟩],
  votesGranted ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}],
  votesResponded ↦ [r1 ↦ {}, r2 ↦ {"r3", "r4"}, r3 ↦ {}, r4 ↦ {}]
]
⟩
```